
SEMA: WHEN THE HASH IS THE WORD

A PREPRINT

Henrik Westerberg

Emergent Wisdom

henrik.westerberg@emergentwisdom.org

April 7, 2026

Revised: August 15, 2026

ABSTRACT

Autonomous agents often reason and coordinate through labels that do not reveal which external information they denote. We present Sema, a content-addressed reference system for reasoning and communication that creates what we call *verifiable words*: participants encode and hash information under an agreed representation, then reuse the resulting content address as a term, optionally paired with a human-readable handle. Sema is designed to be extensible across representations: any representation can be used as long as participants agree on how content is encoded, hashed, resolved, and decoded. This paper develops one concrete realization based on canonicalized JSON-compatible objects, field-level Merkle commitments, explicit identity rules, and content-addressed references. Full-reference verification establishes identity of the resolved hashed content, while semantic equivalence, correctness, and compliance remain separate questions.

As a worked application of the Sema primitive, we introduce Pattern Cards, a concrete schema for expressing deliberate, System 2-style reasoning. They express reasoning moves that models often skip as explicit, checkable contracts covering mechanisms, conditions, invariants, typed dependencies, and failure modes. The same content-addressed definitions support coordination: agents can verify that they are working from the same resolved definitions; when enforced in strict mode, the handshake halts on mismatch.

A bootstrap library of 457 cards demonstrates the design. Its contract fields averaged 26.0 times the tokens of short wire references—a transmission-compression result, not an inference-token saving, because definitions were hydrated before use. Illustrative agent demonstrations examine coordination behavior. The same primitive could support a Living Taxonomy of Thought; its utility, evolution, and governance remain open questions.

Keywords Content-Addressed Semantics, Multi-Agent Systems, Verifiable References, Semantic Coordination, Merkle Commitments, Agent Vocabularies, Pattern Cards

1 Introduction

We are entering the era of the *Agent Web*, where autonomous AI systems negotiate resources, execute code, delegate tasks, and make consequential decisions across organizational boundaries. Yet they lack a fundamental capability: the ability to verify the external information behind the terms they share. Five problems compound:

The Paraphrase Problem. Agent A may need to reproduce a lengthy behavioral specification for Agent B, and each paraphrase risks drifting from the last. A reusable reference could carry the identity of that specification more compactly.

The Identity Problem. When two agents use the phrase “coordinate safely,” there is no mechanism to verify that the label resolves to the same external definition. Silent divergence compounds through agent chains where identical labels hide different specifications.

The Hallucination Problem. Large Language Models fabricate references and citations [1], motivating a “fail-closed” architecture where references serve as cryptographic pointers rather than probabilistic generations.

The Trust Problem. RDF and OWL provide shared representation frameworks, but do not by themselves verify that low-trust peers resolved the same external definition.

The Granularity Problem. A generic report that “the coordination failed” provides no diagnostic utility, whereas “the Lock Invariant in StateLock#c9c2 was violated at $t = 3.2s$ ” isolates the exact failure mode. Human natural language evolved for humans; autonomous systems require a vocabulary layer designed specifically for agents.

Natural language gives compact names to only a small subset of reusable procedures because each additional term must be learned and remembered. A resolvable agent vocabulary relaxes that constraint: detailed definitions can be expanded on demand, allowing recurring procedures to become precise, composable terms.

Sema¹ is a communication primitive: hash information under an agreed representation and carry the resulting content address directly in communication. Its agent-coordination instantiation assumes a low-trust environment where agents may hallucinate, drift, or actively deceive. The accompanying bootstrap library is organized not as a passive “Library Catalog” but as a “Constitution for a Digital City,” with Pattern Cards such as `LatticeCommit#488a` and `ReceptivityGate#5667` specifying integrity mechanisms.

This paper applies Sema’s primitive to definitions—a use with precedent in Hawke’s Semantic Definition Hash [3], Trusty URIs for nanopublications [4], Git, IPFS [5], and Unison [6]—and makes readable, content-addressed references function as lexical terms inside natural-language agent messages. Earlier systems use hashes as citations, routing tokens, method names, and infrastructure addresses; Sema’s distinctive move is to make a hash-anchored term part of the agents’ working vocabulary.

Trusty URIs are the closest precedent to the JSON/Merkle realization used here: both hash a canonical representation above the raw serialization bytes, so an identifier can survive reserialization. RDF identifiers and trusty URIs are naming schemes: they label referents—entities, classes, finished artifacts—so that something can be pointed at and, in the trusty case, verified after the fact. Sema likewise brings content-addressed identity into communication; with Pattern Cards, that mechanism becomes a lexicon. Minting a pattern adds a term to the language the agents themselves write in, and that term carries its own conditions of use (invariants, preconditions, failure modes) instead of merely describing an object. The consequence is productive rather than cosmetic: a lexical entry can be resolved directly, divergence surfaces as a different hash, and a compliant handshake can halt on mismatch.

The implementation in this paper recursively canonicalizes JSON-compatible objects and represents behavioral specifications—invariants, preconditions, failure modes, and typed dependencies—as Pattern Cards whose references agents embed directly in ordinary language. An agent writes: “I `Delegate#00fa` this `Task#a850` to you, expecting a `Solution#8d55` that satisfies this `AcceptSpec#37fa`.” Each anchored term is both a readable word and a verifiable content-addressed reference. A change to a hashed canonical field produces a different digest with overwhelming probability, while canonical-equivalent reserialization, edits to excluded metadata, and renaming the object’s own unhashed handle leave that digest unchanged; comparison before action can therefore expose definitional mismatch before coordination depends on it (Figure 1).² The general reference form is:

$$\text{full-reference}_\sigma(\ell, x) = \ell \parallel \sigma \parallel \mathcal{H}_\sigma(\text{Encode}_\sigma(x)) \quad (1)$$

Here $\parallel$ denotes concatenation, ℓ is an optional readable label, and σ names the agreed encoding and hash scheme. Prose may abbreviate the digest as a short wire reference, but collision-resistant verification resolves that abbreviation to the full reference or an agreed full root.

The primitive operation—encode under an agreed representation, then hash—is well-established; canonicalization is needed when multiple serializations should denote the same content. The cumulative effect of the design choices described below is that content-identity verification becomes native to the medium LLMs operate in.

The practical consequence is that a vocabulary of Pattern Cards behaves like an agent framework mixed into English. Because English is malleable, the framework inherits that malleability: agents reach for as much or as little structure as

¹The name “Sema” (Greek *sēma*, “sign”) was chosen in December 2025. An unrelated system also named SEMA [2], addressing multi-turn jailbreak attacks on LLMs, appeared at ICLR 2026. The two systems share only the acronym and operate at different layers (definition-identity verification vs. attack methodology).

²Large context windows ($> 1M$ tokens) do not subsume this mechanism: long contexts suffer from “Lost in the Middle” attention degradation, and two agents both holding a full definition in context still have no way to verify that they hold the *same* one. Pattern references can act as attention anchors and support cross-agent definition-identity checks that a shared window cannot provide.



Figure 1: The content-addressing pipeline used in this paper. The digest is derived from an agreed representation and paired with a human-readable handle; the four-character form is a wire reference, while full-digest or full-root verification remains the security boundary. Other Sema realizations may use different representations and hash schemes.

the moment requires, naming only what needs naming and falling back to freeform prose for everything else. When a coordination situation recurs with new stakes, an agent mints a pattern that captures it; every downstream reference to that situation becomes a verifiable pointer to the captured shape rather than a paraphrase prone to drift. A page of careful prose can often be rewritten as a few sentences that cite the relevant patterns by hash, preserving the definitions of the referenced mechanisms while leaving ordinary contextual prose freeform—Section 7.4 measures this empirically across the bootstrap library. Given a resolver and the full identifier, expansion returns the same verified canonical content.

The same substrate can act as collective memory across summarization. Prose may shift at each compaction boundary, while a preserved full identifier remains pinned to the same verified definition for as long as its content remains resolvable. Sessions and agents can therefore retain stable conceptual anchors rather than relying only on surviving phrasing.

The reasoning contribution of Pattern Cards depends on how they are used. When a model already holds a concept precisely, re-supplying the definition adds tokens without improving accuracy. The contribution instead runs through three channels: verified reference identity, which lets a conclusion cite the exact external definitions it used and exposes mismatch before work proceeds from a false shared assumption; checkable invariants available mid-task, where a card’s contract functions as a checklist the agent can test its work against; and names for procedures that practitioners use but rarely write down.

Models already produce fluent reasoning chains, and their fluency can allow unchecked premises to pass review. Pattern Cards are intended to make the steps explicit and enforceable by a surrounding workflow: citable by hash, checkable against a contract, and usable as inputs to a gate. The intended benefit lies in multi-step work, coordination, and long-horizon tasks rather than single-pass scoring over well-known material. Some of that benefit falls outside answer-only metrics: a verified refusal prevents an action or assertion rather than producing a scored answer.

The cognitive-science framing is direct. Documented failures of human judgment—base-rate neglect, survivorship bias, unchecked confirmation—often persist even when the relevant corrective procedure is known. Checklists and structured analytic techniques make those procedures easier to invoke. A pattern vocabulary extends that approach by making the moves named, composable, verifiable, and cumulative. The bootstrap library treats this vocabulary as a reusable standard library for deliberate reasoning.

Many human reasoning conventions remain implicit because readers are expected to supply them. For language models operating on text, omitted steps are unavailable unless learned during training or supplied through context. A vocabulary of explicit, reusable reasoning procedures makes those moves available as contracts rather than leaving them implicit in prose.

This coordination use is the first operational slice of a larger architecture: a Living Taxonomy of Thought. A Sema-equipped model could represent a book, theory, or strategy as a graph of referenced concepts, mechanisms, assumptions, and disagreements. Where an existing pattern fits exactly, the model reuses it; where the fit is only partial and the difference changes the concept’s contract, it mints a linked variant rather than overloading the old word. Successive agents can therefore inherit not only documents, but an inspectable record of the conceptual distinctions extracted from them. Sema supplements rather than replaces the source material, and the resulting taxonomy need not be final: source-specific patterns may remain local, competing interpretations may coexist, and only broadly reusable concepts need enter a shared library. This paper implements the identity, resolution, composition, and minting substrate for that architecture; training models to construct and reason through work-scale pattern graphs remains future work.

2 Problem Formalization

We use a minimal communication model to isolate the failure mode that motivates Sema; we do not attempt to formalize semantic equivalence in general. Let $\mathcal{A} = \{A_1, \dots, A_n\}$ be a set of agents, where each agent A_i maintains a local semantic model $M_i : \mathcal{L} \rightarrow \mathcal{S}$ mapping linguistic tokens $\mathcal{L}$ to internal semantic representations $\mathcal{S}$. When Agent A_i sends message $m \in \mathcal{L}$ to Agent A_j , coordination is reliable only if:

$$M_i(m) \approx_{\epsilon} M_j(m) \quad (2)$$

where $\approx_\epsilon$ denotes semantic equivalence within tolerance ϵ . The failure mode is that agents cannot directly compare their private semantic mappings M_i and M_j , so divergence accumulates silently until coordination breaks. Sema does not prove equality of internal representations. In the definition-coordination application developed here, full-reference verification instead establishes that agents resolved the same hashed external definition, independent of their private internal models.

A coordination mechanism for external semantic contracts must satisfy five key desiderata. *Identity* (D1): agents can test whether two references denote the same canonical definition. *Fail-Closed Coordination* (D2): mismatch blocks action rather than proceeding with different interpretations. *Compactness* (D3): for nontrivial definitions, wire references should be smaller than the content they denote. *Resolvability* (D4): a reference can be expanded or verified against the canonical object, without loss with respect to that canonical object (though not with respect to whatever private meaning an agent additionally attaches to it). *Permissionless Minting* (D5): no central authority controls vocabulary creation.

3 The Sema Protocol

The preceding section established that current agent coordination lacks a mechanism for verifying shared external definition identity. This section first specifies Sema’s representation-agnostic communication primitive, then describes the JSON/Merkle realization used in this paper and its agent-coordination layers: the Pattern Card structure (Section 3.3), the fail-closed handshake (Section 3.4), and the vocabulary architecture used by the bootstrap library (Section 4).

3.1 Design Principles

Sema’s core rule is representation-agnostic: hash information under an agreed representation and scheme, then use the resulting content address directly in communication. It is not restricted to JSON or to a fixed schema. Raw bytes, a canonical graph, or a structured object can each participate if recipients know how to resolve the reference and recompute its identity. The implementation used in this paper supplies deterministic recursive Merkle hashing for JSON-compatible objects; a molecular database, legal contract repository, or game-rule engine could define a different representation while retaining the same communication primitive.

The communication primitive, the JSON/Merkle realization, the Pattern Card schema, and the *Sema Bootstrap Library* are distinct layers. Pattern Cards are this paper’s concrete agent-coordination schema; the library in Section 4 is a bootstrapped opinion on useful definitions expressed through that schema. Coordination requires a shared Schelling point, so we provide 457 patterns designed to bootstrap agent society. Their fields describe actionable behavioral contracts rather than passive entries: the `mechanism` states the intended procedure, the `invariants` state what must remain true, and the `dependencies` state how a pattern composes. Execution and enforcement still require an agent, evaluator, implementation binding, or surrounding harness. What follows describes the JSON/Merkle mechanics and this concrete instantiation.

3.2 Cryptographic Foundation

Applied to definitions, the communication primitive permits a *Semantic Commons*: a shared namespace where agents actively mint and reference new concepts without requiring a central authority to decide what words exist. Just as Git allows any developer to commit code, Sema allows any participant to mint a content-addressed item under an agreed representation. In this implementation, an agent defines a Pattern Card as a JSON object and computes its Merkle Root from the card’s semantic fields, transforming the namespace from a read-only reference into a writable *thoughtspace*.

$$a_\sigma(x) = \text{Hash}_\sigma(\text{Encode}_\sigma(x)) \quad (3)$$

Here σ identifies the agreed representation and hash scheme. Sema’s general communication primitive carries $a_\sigma(x)$ as a reference; it does not require a particular encoding. The JSON/Merkle implementation specializes this rule to field-addressable Pattern Cards:

$$H_{\text{pattern}} = \text{Merkle}(\{\text{mechanism, gloss, dependencies, } \dots\}) \quad (4)$$

This hash becomes the permanent identity component for that object (e.g., `#a1b2`); the readable handle remains separate. If Agent A and Agent B resolve the same full identifier, recompute the hash over the returned object, and obtain a match, they establish—subject to SHA-256 collision resistance—that they hold the same canonical definition, regardless of who created or stored it.

The JSON realization implements content-addressing via a Merkle Tree. Given a definition d and a separate readable label ℓ , the Pattern Card identifier is:

$$\text{id}(\ell, d) = \text{sema} : \|\ell\| \# \text{mh} : \text{SHA-256} : \|\text{RootHash}(d)\| \quad (5)$$

where ℓ is not part of $\text{RootHash}(d)$, and $\text{RootHash}(d)$ is computed via recursive Merkle Tree construction using deterministic JSON canonicalization inspired by RFC 8785 (JSON Canonicalization Scheme) [7], adapted with explicit NFC string normalization for deterministic string equivalence, and aligning with principles from RDF Dataset Canonicalization [8] and content-addressed semantic data [9]. This ensures that canonical-equivalent JSON objects produce deterministic hashes regardless of serialization order.

The current implementation uses canonicalization v2 (semahash 0.3.0), which domain-separates every Merkle node with an explicit type tag. Strings are hashed as $\text{SHA-256}(s : \|\text{NFC}(\text{collapsed text})\|)$; primitive JSON values as $\text{SHA-256}(p : \|\text{canonical JSON}\|)$; lists as $\text{SHA-256}(l : \|\text{H}(L_1) \|\text{H}(L_2) \|\dots\|)$; and dictionaries as $\text{SHA-256}(d : \|\text{H}(k_1) \|\text{H}(v_1) \|\dots\|)$, with entries sorted by normalized key and with normalized-key collisions rejected fail-closed. Dependency maps receive one additional canonicalization step: aliases are normalized through their target handles, and multiple aliases that intentionally reference the same handle are preserved as a sorted list rather than silently collapsed. These tags prevent structurally distinct JSON values—for example the string "1" and the number 1, or a two-element list and a one-entry dictionary—from sharing the same hash input.

Vocabulary snapshots require a second canonicalization layer because a set of disconnected pattern DAGs has no traversal-defined order. Beginning with semahash 0.4.0, the implementation therefore publishes two versioned commitments. The *semantic-set root* commits to the sorted set of unique raw pattern digests; the *catalog root* commits to the sorted mapping from each handle to its raw pattern digest. Both use scheme-specific leaf framing followed by the Merkle Tree Hash construction of RFC 9162 [10]: leaves are $\text{SHA-256}(0x00 \| d)$, internal nodes are $\text{SHA-256}(0x01 \| L \| R)$, and a non-power-of-two list is recursively split at the largest power of two below its length. Thus an unpaired subtree is never duplicated. Sorting raw digests is the rule used here for unordered-set semantics—Certificate Transparency itself receives an already ordered log—while catalog entries are ordered by their handle bytes. The scheme identifier and leaf count accompany each root, because a digest alone identifies the hash function but not the tree construction.

The per-pattern Merkle structure assigns every field a deterministic, collision-resistant hash, allowing agreement and disagreement to be localized even when full pattern roots differ. As a concrete example, suppose Agent A and Agent B both reference `StateLock#c9c2` but their pattern roots disagree. Field-level hash comparison may reveal matching invariants hashes (both definitions state that the lock is exclusive and auto-releases on timeout) while their `failure_modes` hashes diverge (A’s pattern enumerates network partition as recoverable; B’s treats it as terminal). This identifies precisely which clauses match and which require reconciliation. Selective coordination on a matching subset requires an explicit policy; the current handshake remains fail-closed on root mismatch. Whole-document hashes (SDH, Agora, BlockA2A) collapse the comparison to a single binary mismatch, whereas the Pattern Card Merkle structure preserves field-level diagnostic granularity.

Crucially, the Pattern Card schema excludes metadata from identity. The `_meta` object, containing fields like `path` (an ordered list of taxonomy segments such as `["Society", "Governance"]`), `tier`, `ring`, and `related`, is excluded from the hash calculation. This design prevents rigid taxonomies by separating two fundamentally different concerns: the mechanism (what a pattern does) is mathematics, immutable and hashed, while the taxonomy (where it belongs) is politics, mutable metadata subject to community consensus. This separation allows the community to reorganize categories, for instance by downgrading a pattern from Tier 1 to Tier 2 upon discovery of a vulnerability, without breaking the code of agents that rely on the pattern’s hash; the identifier `BayesUpdate#11d2` remains valid even if its classification changes. The pattern identity is computed from eleven semantic fields:

Field	Purpose
<code>mechanism</code>	How the pattern works (the core definition)
<code>gloss</code>	Short definition (one-line summary)
<code>dependencies</code>	References to other patterns (the wiring)
<code>signature</code>	Type transformations, e.g., <code>Gate(Signal)</code>
<code>invariants</code>	Rules that must always hold
<code>preconditions</code>	Required state before invocation
<code>postconditions</code>	Required state after a valid invocation
<code>parameters</code>	Configurable parameter objects
<code>failure_modes</code>	Known ways the pattern can fail
<code>data_schema</code>	JSON Schema for runtime data structure
<code>extends</code>	Exact parent definition this specializes

Fields not hashed include a pattern’s own handle, `_meta`, and any `sema_*` fields. Reclassification therefore leaves identity unchanged, and renaming a pattern does not change that pattern’s own digest. Canonicalization v2 does, however, retain target handles inside hashed structured references: dependency maps contain target handles, and

`extends` stores a full Sema ID. Renaming a referenced pattern can therefore change the hashes of both dependents and specialized descendants; graph-wide rename invariance requires a separate versioned canonicalization migration covering both forms. For backward verification, a pre-0.4 card carrying the former `derived_from` key is hashed under that original key and is not reinterpreted as an `IS_A` claim; explicit migration to `extends` produces a new identity.

For human readability, prose references use a 4-character stub of the root hash, such as `PropheticQuorum#fb6b`; full dependency fields and security-sensitive handshakes retain or resolve the full hash or compare a vocabulary root. To ensure that logic remains stable even when dependencies evolve, the implementation uses a template hashing strategy where the mechanism uses local placeholders like `{{hypothesis}}` representing the pure algorithm, while the wiring maps these placeholders to specific hashes in the `dependencies` object. The Pattern Identity is the Merkle Root of both, allowing tools to distinguish a logic-field match from a dependency-wiring mismatch when agents use different library versions.

3.3 Discovery and Pattern Cards

The Pattern Card tooling reconciles two opposing needs: discovery, which benefits from ambiguity and high recall, and coordination, which requires exact identity for the references being verified. It does so through *progressive ambiguity collapse*, formalized as the `Taper#bebf` pattern, where the discovery lifecycle follows a strict sequence of decreasing entropy. First, in the *Orient* phase, the agent queries the topology via `sema_graph_skeleton()` to obtain a low-resolution map of regions and hubs. Next, during *Explore*, the agent performs a hybrid search merging field-weighted keyword matches (handle: 1.0, signature: 0.75, gloss: 0.70, mechanism: 0.55) with vector embeddings (Score 0.3–0.7). This strategy mirrors Blended RAG [11], merging keyword precision with semantic recall [12]. However, reliance on fuzzy search alone is hazardous; audits of encyclopedic search engines reveal that they frequently surface content only “weakly related” to the query [13]. This hybrid strategy allows approximate intent (e.g., “how to handle errors”) to find precise patterns (e.g., `Retry#4f9f`), but necessitates the subsequent verification step. Finally, in the *Verify* phase, the agent invokes `sema_handshake()` (Section 3.4), which distinguishes cooperative drift detection from strict identity verification. The pipeline uses semantic similarity for discovery and hash-based identity comparison for coordination.

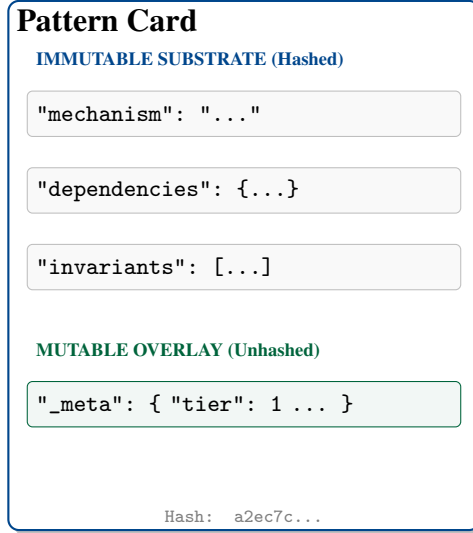


Figure 2: Pattern Card Anatomy. The *Substrate* defines identity (hashed); the *Overlay* enables evolution (unhashed).

A Sema Pattern is not merely a dictionary definition; it is a structured behavioral specification (Figure 2). Preconditions, postconditions, and invariants make contract clauses explicit for inspection by agents, external evaluators, or executable bindings; the Pattern Card does not itself execute or enforce them. This structure builds on a lineage of structured LLM frameworks—notably DSPy Signatures [14], DSPy Assertions [15], and the Prompt Pattern Catalog [16]—which introduced contract-like constraints for language model pipelines. Sema’s contribution is to make canonical contract identity hash-verifiable: including the clauses in the Merkle root permits cached digest comparison without asking a model whether two contract texts mean the same thing. We selected these fields to transform vocabulary from *Passive Knowledge* into explicit contract structure, as illustrated by the schema for `PropheticQuorum#fb6b`:

Dependencies (The Imports) Rationale: A categorized map of hard links. Makes declared dependencies machine-readable without parsing natural language.

Example:

```
"dependencies": {
  "composes_with": {
    "sim": "sema:Simulation#mh:SHA-256:1058
           e7090079e846251ed480d7afec0328809d22bebb6b205425b2de614787c3",
    "vote": "sema:Vote#mh:SHA-256:84935
            dbc846bcd8fe2da01e9e04401f190769e87d0354e00c0986e43e414f300"
  },
  "accepts": {
    "proposal": "sema:Proposal#mh:SHA-256:5
                d0a98055eef6c963d4c9ec0403cac243720ce8927398032c19b4f5d8cc94506"
  },
  "yields": {
    "decision": "sema:Decision#mh:SHA-256:
                b1407127acd4a39314fc24b418310e4b95e0347e4e22f376151ab765e2dfd2c7"
  }
}
```

Mechanism (The Procedure) Rationale: Describes the intended procedure using the keys defined in Dependencies.

Example: “Instantiates `{{sim}}` to predict outcomes of `{{proposal}}` before calling `{{vote}}` to produce `{{decision}}`.”

Invariants (The Safety Contract) Rationale: States conditions that must hold throughout a valid application; checking them requires an agent, evaluator, or executable binding.

Example: “Prediction Precedence: Vote must occur after Simulation.”

Preconditions (Entry Conditions) Rationale: Records conditions that must hold before application, supporting chaining and planning when an agent or external evaluator checks them.

Example: “Determinism level is high.”

Gloss (The Embedding Anchor) Rationale: A semantic hook for vector retrieval.

Example: “Aligning predictions before aligning votes.”

Specialization (The Type Relation) Rationale: Records that this definition is a narrower kind of an exact parent definition and emits an IS_A edge.

Example: “extends”: “sema:Task#mh:SHA-256:...”. Because the parent version is named by its full identifier, retargeting the child is an explicit semantic change.

Metadata (The Overlay) Rationale: Stores the evolving knowledge graph (associations, suggestions, safety tiers) without affecting the identity hash.

Example: “_meta”: { “tier”: 1, “related”: [“Mutex#b031”], “supersedes”: [“sema:OldHandle#mh:SHA-256:...”] }. The unhashed `supersedes` list records version history without making the route by which a definition was authored part of its identity.

3.4 The Fail-Closed Protocol

When agents coordinate using Sema patterns, the protocol provides a fail-closed handshake for comparing selected external-definition identities. The handshake is available as an MCP tool (`sema_handshake`) that agents invoke before coordination; enforcement depends on the agent’s execution harness calling the tool, not on the protocol itself. Rather than verifying every pattern individually, agents select a relevant subset of patterns (the “Context”) required for the current interaction and deterministically construct the catalog Merkle Tree of this subset. Each leaf binds a requested handle to the hash of the definition it resolved, so swapping two handles’ definitions cannot preserve the root. The initiating agent proposes a context set by listing the required pattern identifiers; the receiving agent independently resolves each identifier, constructs the same Merkle Tree, and proceeds only if both the declared scheme and full roots match. The agents then exchange the Merkle Root of their respective trees:

$$R_{context} = \text{CatalogMerkleRoot}(\{(h_1, \mathcal{H}(d_1)), (h_2, \mathcal{H}(d_2)), \dots, (h_n, \mathcal{H}(d_n))\}) \quad (6)$$

If the full roots and schemes match, agents have a collision-resistant commitment that every selected handle resolved to the same canonical definition—and therefore share an identical external semantic contract for each—allowing them to proceed with optimized, short references. If the roots or schemes differ, the system detects divergence and halts immediately. This ensures that *compliant* agents never proceed with subtly different baseline definitions of the instructions, effectively detecting divergence before coordination begins (Figure 3). The comparison must travel through an authenticated channel if an active attacker is in scope. The mechanism is not a Byzantine guarantee: it assumes participation in the handshake and therefore guarantees agreement only for compliant agents.

The current convenience tools `sema_propose_context` and `sema_verify_context` exchange an eight-hex-character catalog-root prefix and label it explicitly as cooperative drift detection, not adversarial security; they require the declared scheme even in this mode. Deployments requiring collision-resistant identity assurance compare the full catalog root under that scheme through an authenticated channel. A full semantic-set root answers the weaker question of whether the raw definition-digest sets match; under canonicalization v2, those digests can still change when target handles in structured references are renamed.

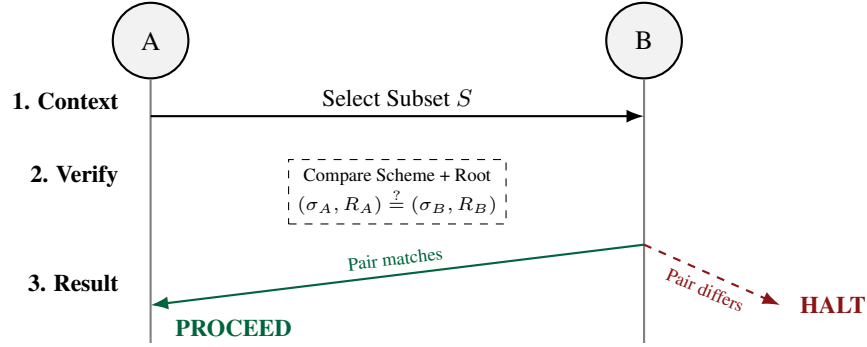


Figure 3: Fail-closed handshake. Agents compare the root scheme and Merkle Root of the agreed pattern subset; an enforced handshake maps divergence in either to HALT.

This represents a deliberate inversion of Postel’s Law [17]. Where the Internet’s robustness principle advises “be liberal in what you accept,” Sema inverts the receiving side: be strictly conservative in what you accept, and reject what you cannot verify. This choice prioritizes safety over permissive interoperability: compliant deployments reject mismatches in the selected external definitions rather than interpreting them on a best-effort basis. In multi-agent systems where definitional drift can cascade into consequential failure, an explicit fail-closed boundary is valuable.

Fail-closed behavior holds only in compliant deployments: an agent that never calls `sema_handshake` can coordinate on any definition it wishes. In cooperative mode, an invoked prefix handshake provides drift detection; a strict handshake comparing full digests or roots rejects divergent external definitions that would otherwise hide behind shared labels. In adversarial deployments that require handshake tokens, a non-compliant agent’s messages are rejected at that boundary, and the comparison must use authenticated references or transport. Enforcement at the coordination-protocol level—making handshake calls mandatory in the agent’s execution harness rather than merely available—is an integration decision beyond the scope of the Sema protocol itself, and is discussed in Section 9.

Proposition 1 (Canonical Hash Collision Bound). *Under the standard random-oracle approximation, for a vocabulary of size $|\mathcal{V}|$ the probability of two distinct canonical definitions producing the same SHA-256 digest is bounded by:*

$$P(\text{collision}) \leq \frac{|\mathcal{V}|^2}{2^{257}} \approx 0 \quad (7)$$

This follows directly from the collision resistance of SHA-256. With $|\mathcal{V}| = 457$, collision probability is negligible ($< 10^{-72}$).

Proposition 2 (Strict Fail-Closed Detection). *If agents A_i and A_j resolve distinct canonical definitions d_i and d_j for pattern P , then $\mathcal{H}(d_i) \neq \mathcal{H}(d_j)$ with overwhelming probability. A strict, enforced handshake comparing full digests or full roots therefore halts for compliant agents; cooperative prefix mode provides drift detection only. Adversarial deployments additionally require authenticated references or transport.*

Proposition 3 (External Contract Identity). *If agents A_i and A_j both resolve a full identifier I against an honest content-addressed store, obtain the same canonical definition d , and verify $\mathcal{H}(d) = \text{digest}(I)$, then they hold an identical external semantic contract for I , independent of their private internal semantic models M_i, M_j . Sema does*

not prove $M_i(I) = M_j(I)$; subject to hash collision resistance, it establishes that the externally resolved object is the same, so any divergence in downstream behavior cannot be attributed to the agents having resolved different definitions under that identifier.

Proposition 4 (Compression with Resolvable Expansion). *Let $|r|$ be reference length (handle + 4-char hash ≈ 15 characters) and $|d|$ be definition length. Compression ratio is $|d|/|r|$. Given a declared resolver or catalogue in which r maps unambiguously to d , expansion is deterministic. Cryptographic assurance still requires verification against the full pattern hash or an agreed context/vocabulary root; the short stub is a readable locator, not a self-authenticating security boundary.*

3.5 The Type System

This subsection describes the type system of the Sema Bootstrap Library. It is a *particular* type system, not *the* type system: the protocol above only requires an agreed representation and hash scheme. The JSON/Merkle realization likewise does not require the Pattern Card schema: different communities can define different JSON schemas, and other implementations can use different representations entirely. Their references remain content-addressed, but field-level interoperability and resolution depend on shared schemes; the current registry and handshake tools implement the Pattern Card case. In this sense Sema is closer to Unicode than to a universal language: it standardizes a way to carry identity, not every vocabulary spoken through it.

We chose the agent-protocol schema below because structural typing lets the same verb operate across domains as long as subjects satisfy a common contract, and polymorphic dispatch lets an agent request “any pattern that implements Deep(Discover)” rather than pinning a specific hash. Readers interested only in protocol-level guarantees may skip to Section 4; readers interested in how this schema handles polymorphism, composition, and wiring across 457 patterns should continue here.

A central tension in vocabulary design is determining when a change in attributes constitutes a change in identity. The bootstrap library adopts a structural resolution: qualitative differences create distinct patterns, while quantitative differences create parameters with hashed range contracts. For example, `ExponentialBackoff#f5e5` with a `base_delay` of 100ms and the same pattern with a 5-second delay share the same Pattern-definition identity because the hash captures the parameter range contract, not the instance value. However, if changing a variable alters the failure mode, such as switching from busy-waiting to database storage, it becomes a distinct pattern (e.g., `SpinLock` vs. `Lease`). This declares a range against which a surrounding runtime may validate instance values while preserving one pattern definition.

To support open-ended reuse without creating incompatible ad hoc interfaces, the Pattern Card schema employs Typed Interfaces rather than named fields. The `dependencies` object is strictly partitioned into four mutually exclusive categories, creating a directed acyclic graph (DAG): `accepts` defines passive input data patterns consumed by the mechanism; `yields` defines passive output data patterns produced by the mechanism; `composes_with` defines active tool or logic patterns explicitly invoked by the mechanism; and `references` defines patterns used for conceptual clarity but not execution. This partitioning enforces a “Single Source of Truth” where every semantic link lives in exactly one place, and decouples verbs from specific noun instances. A `LogisticsAgent` locking a shipping container and a `MedicalDrone` locking a patient record both invoke the same pattern because both subjects satisfy the `accepts` interface (e.g., `"accepts": { "subject": "sema:UniqueHandle..." }`). Compatibility depends on declared types rather than variable names. This architecture allows the 457 core patterns to support many cross-domain compositions through shared noun definitions.

The integrity of these compositions is enforced by the Explicit Wiring Rule, which welds the abstract type to the concrete logic. The compiler mandates that every element in a pattern’s signature (the claim, e.g., `Check(Safety)`) must be explicitly imported in the dependencies and invoked in the pattern’s text fields (mechanism, invariants, preconditions, postconditions, or failure modes) via a template key. An accepted Pattern Card therefore cannot declare a signature without the required links in its hashed specification; this validates the card’s wiring, not the behavior of an eventual implementation.

The Pattern Card compiler resolves these high-level intents into concrete low-level pattern handles. When an agent submits a polymorphic intent such as `Deep(Discover)`, the compiler queries the vocabulary graph for a pattern declaring that signature (e.g., `DeepResearch#c1f1`) and resolves the dependency chain. This resolution step decouples intent from a specific pattern definition, allowing the system to retarget the underlying procedure, such as swapping a heuristic check for a rigorous crypto-audit, without altering the agent’s cognitive script. Executing the resolved procedure still requires an agent or implementation binding.

At the wire level, the hash functions as a word in the natural-language stream that agents exchange; the LLM does not reason directly over hash bytes. To use the referenced procedure, an agent or host runtime resolves the definition and

Table 1: The Deep Registry: Resolving Polymorphic Rigor. Fast and Slow refer to lightweight versus heavyweight resolution strategies.

Intent (Query)	Source Pattern (Fast)	Resolved Target (Slow)
Deep(Heuristic)	HeuristicSnap#b95f	MentalSim#1ca0
Deep(Trace)	Trace#b68d	GenealogicalTrace#ee20
Deep(Discover)	Discover#f6e0	DeepResearch#c1f1
Deep(Nature)	Nature#9ac1	LivedProof#4aa1

places the relevant text in model context. The deployment pattern assumed here is:

$$\text{Prompt} = \text{System} \oplus \text{Hydrate}(\text{Patterns}) \oplus \text{Query} \quad (8)$$

The *wire-level token* is simultaneously a readable handle (PropheticQuorum#fb6b) and a pointer into a cryptographically verifiable definition space: when the resolver checks a full hash or agreed root, a change to hashed canonical content produces a detectably different definition identity. Unlike infrastructure-oriented file addresses, these Pattern Card references are designed to be read as lexical terms in the medium agent communication already uses. The short stub reads fluently; collision-resistant assurance comes from resolution and full-digest verification. This design trades context-window tokens for wire-level integrity.

4 The Bootstrap Library

This section presents the Sema Bootstrap Library: the Schelling-point vocabulary introduced in Section 3. The library is an initial seed rather than a closed canon; if an agent requires a different locking mechanism, it can permissionlessly mint a distinct variant such as LeaseStateLock without breaking the protocol.

4.1 Methodology

The initial vocabulary is a pragmatic bootstrap assembled without a principled selection procedure. A different author could produce a different vocabulary using the same protocol; the protocol’s value is independent of any particular library. The 457 patterns emerged iteratively from four sources: distributed-systems primitives, the cognitive-decomposition architecture developed in [18], model-assisted gap analysis, and the author’s domain analysis of agent failure modes. Creativity protocols such as Adversarial Evolutionary Generation [19] were used for some frontier patterns.

The process became more structured as the library grew. Refinement checkpoints used a dedicated “Taxonomist” role to reject duplicates, merge overlapping concepts, and enforce the Noun/Verb separation; repeated failures became rules and guidelines (Section 4.8); and the database layer enforces DAG acyclicity, dependency-hash transitivity, bidirectional wiring checks, and Rule-G layer direction. The atomic `sema apply` step operationalizes these constraints by validating additions and removals before mutation, catching handle-format errors, signature mistakes, layer-direction violations, near-duplicates, and rigor warnings on every mint and CI pass.

Minting is gated by two criteria. A concept earns a pattern if it requires *protocol consistency*—multiple agents must coordinate on the exact semantics, as with `StateLock#c9c2`—or if specifying it produces *structured thinking*, forcing a loosely used English concept into an invariant-bearing mechanism. English suffices when neither criterion is met. Pattern *generality* is then checked by a broad-use test: the mechanism and invariants must capture what every legitimate deployment context needs, while context-specific features become descendant territory via *extends* (e.g., `Lock#95c2` → `Mutex#b031`, or `Task#a850` → `BoundedTask#86dd`). The test prevents both overfit mechanisms that break on legitimate variants and toothless abstractions that constrain nothing.

After the refinement pass described in Section 4.8, the library has no duplicate mechanisms or semantic collisions detected by the applied audits (hash collision, canonical-form equality, and embedding-similarity sweeps above a threshold) across 457 patterns. These audits and Taxonomist review maintain quality, but they do not guarantee functional uniqueness (see §9, “The Synonymy Limit”). We make no claim to completeness or optimality.

4.2 The Civilization Stack

The vocabulary is not a flat list; it is structured into four fundamental layers mimicking a civilization stack. At the base lies **Physics**, substrate primitives that obtain independently of any author (e.g., `Lock#95c2`, `Entropy#d5f7`, `Gradient#ecfc`, `Equilibrium#4f03`, `Conservation#0b32`, `Distance#dfdf`, `PhaseTransition#b775`, `Attractor#0036`,

MutualInformation#e714, and Measurement#511d). Above this rests **Mind**, mechanisms that structurally require cognition (e.g., BayesUpdate#11d2), where judgment must be exercised and a single isolated agent can execute the mechanism. Emerging from these is **Society**, mechanisms that structurally require ≥ 2 independent parties with potentially divergent state (e.g., Consensus#b9a5). Enclosing all three is **Infrastructure** (Figure 4): authored structures and operations that do not require cognition to execute—data types, composite topologies, and mechanical operations (e.g., Mutex#b031).

The placement of each pattern is governed by the *mechanism-sufficiency test*: what does the pattern’s mechanism structurally require to execute? A pattern whose mechanism requires nothing more than substrate goes to Physics; one that requires authored structure but no cognition goes to Infrastructure; one that requires cognition but can be executed by a single isolated agent goes to Mind; one that structurally requires another party with separate state goes to Society. Placement follows the mechanism’s execution requirements rather than its typical application domain or subject matter. This test is the governing principle recorded in docs/core/philosophy.md §3.1.

The four-layer ordering is a design discipline for this bootstrap library; other semantic vocabularies may use different decompositions. Its motivation is operational rather than ontological: lower layers should not depend on higher layers because doing so makes mechanical guarantees depend on social or cognitive assumptions. Resource and execution guarantees should not depend on reasoning strategies or social protocols, while social protocols may safely depend on lower-level execution and reasoning primitives. The “Semantic Inversion Attack” analyzed in §7.5 is the concrete failure mode this discipline prevents.

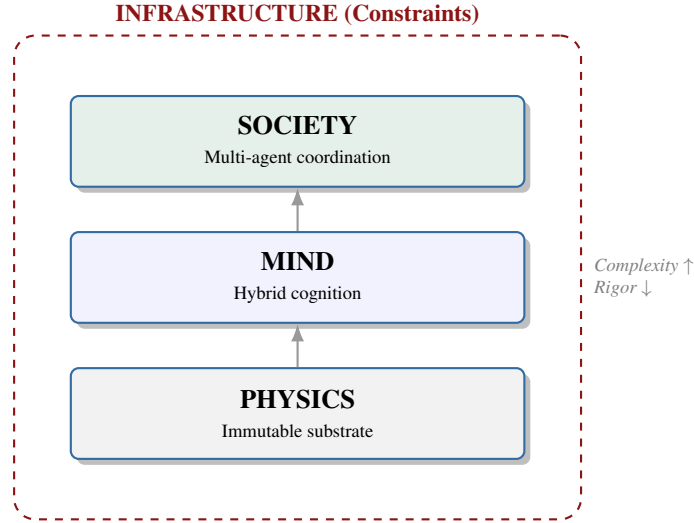


Figure 4: The Civilization Stack. Infrastructure constrains the stack; Physics supports Mind, which enables Society. Arrows indicate dependency: upper layers rely on guarantees from lower layers.

This stratification enforces a property similar to Stewart Brand’s “Pace Layering” [20]: lower layers provide stability for the turbulence of upper layers. Separating **Physics** primitives such as Entropy#d5f7 from **Society** agreements such as Constitution#3deb allows social contracts and voting protocols to change without redefining lower-layer terms. The layer discipline therefore reduces coupling between governance and substrate guarantees.

To prevent semantic inversions, the dependency graph must form a DAG: the validator rejects cycles because recursive dependency hashing would otherwise be undefined. Because layer metadata is excluded from the hash, the communication primitive itself is agnostic to layer choice: a card’s identity depends on mechanism and dependencies, not on a library’s filing tier. Libraries may therefore adopt different layer policies while retaining hash-level interoperability when they share the Pattern Card scheme.

The bootstrap library applies a stricter policy (Section 4.8): hard dependencies (accepts, composes_with) must flow from higher layers to lower (Society $\rightarrow$ Mind $\rightarrow$ Infrastructure $\rightarrow$ Physics), enforced as a sema apply gate, while soft citations (references) and outputs (yields) are exempt and allow semantically necessary cross-layer links (for example, Belief#e160 in Infrastructure soft-references Agent#e127 in Mind because beliefs are conceptually held by agents, and the reference is acyclic). The current vocabulary contains 457 patterns organized into 12 categories across these four layers (Table 2; complete taxonomy in Appendix B), with zero patterns violating the layer-direction rule at this version.

Table 2: Bootstrap library: 457 patterns across 12 category names and 4 layers

Layer	Category	N	Examples
Physics	Primitives	15	Attractor, Causation,
	Time	1	Conservation CausalBarrier
Infrastructure	Data Structures	96	AcceptSpec, Aesthetics,
	Primitives	53	Anomaly
	Verification	9	Act, Actor, Aggregate AuditTrail, CompatibilityCheck, ExplainBeacon
Mind	Inference	22	BaseRateInclude, BayesUpdate, BreadthGovernor
	Memory	15	BeliefTracking, ChunkMerge, ContextCompress
	Reasoning	63	Abduction, Axiom, BackwardChain
	Strategy	81	AdversarialSteel, Agent, AnalogyBridge
Society	Coordination	12	Compromise, Consensus, ConsensusFinder
	Economics	9	AtomicBid, Award
	Governance	7	AttentionMarkets, AnchorDrop, Constitution, DocumentedOverride
	Protocols	74	AdversarialProof, AgentDiscover, AgentProtocol
Total		457	

4.3 The Grammar of Agency

The vocabulary is not a flat list of tools but a compositional grammar that enables agents to construct novel intents from atomic units. Ring 0 *Primitives* act as the fundamental building blocks of agent cognition. Five foundational verbs map to dimensions of autonomous inquiry: *Discover#f6e0* (Breadth) acts as the Scout, querying the horizon to find new nodes, agents, or paths; *Deep#16b0* (Depth) serves as the Scientist, escalating from heuristic proxies to rigorous implementations; *Trace#b68d* (Time) functions as the Historian, reconstructing causal chains within the records it holds; *Check#6890* (Validity) operates as the Critic, filtering inputs against invariants; and *Stigmergy#dd4f* (Space) works as the Builder, writing persistent signals for indirect coordination. Alongside these verbs, noun primitives such as *Belief#e160*, *Agent#e127*, and *Constraint#7013* provide the objects of agency.

These primitives enable a higher-order grammar where complex behaviors compose from atomic units: *Check#6890*(Condition) evaluates a target *Condition#b480* and yields *Verified*, *Falsified*, or *Unknown* without changing control flow; *Trace#b68d*(Target) wraps a process to generate an immutable lineage log whose coverage may be partial; and *Stigmergy#dd4f*(Signal) marks the shared environment with a signal for others to discover.

Ring 2 *Macros* are composite patterns built from primitives via semantic import. (Ring 0–2 describes compositional complexity; Tier 1–3, Section 4.5, describes safety rigor. Both are stored in the unhashed metadata overlay.) For example, *AgentDiscover#b784* imports *{{Discover#f6e0}}* acting on *{{Agent#e127}}*, while *TraceBelief#7ccc* imports *{{Trace#b68d}}* acting on *{{Belief#e160}}*. This grammar allows agents to express intent via queries like “I need to *Deep*(*Trace*) this belief,” which the runtime resolves to concrete pattern definitions.

This distinction reveals a bicameral architecture. Primitives serve as the query interface, allowing agents to identify logical gaps in the vocabulary. Macros like *IdentityHandshake#4397* serve as the social interface, enabling efficient coordination through compression. Instead of negotiating four separate primitives in four round-trips, agents verify a single Macro hash in one. Primitives are for querying; Macros are for talking.

The verb/noun distinction introduced in the Type System (Section 3.5) also addresses the “schema drift” problem where implicit schemas lead to subtle incompatibilities. Pattern Cards formalize the “shape of work” as content-addressed Data Patterns. `Task#a850` represents a recursive definition of intent that inherits constraints from its parent and explicitly defines its `AcceptSpec`. `Solution#8d55` acts as a container for work product, encapsulating provenance and, for composite work, a component tree for supply-chain auditing. `UniqueHandle#1853` specifies a rivalrous resource pointer under linear logic: a conforming runtime treats transfer from Agent A to Agent B as revoking Agent A’s access. To ensure structural compatibility, Noun definitions may include a validation schema in the `data_schema` field (e.g., `Task#a850`, `Bid#79cf`), embedding constraints directly into the identity unlike external standards such as SHACL [21]. This allows agents to construct rigorous semantic sentences:

“I Delegate#00fa this Task#a850 to you, expecting a Solution#8d55 that satisfies this AcceptSpec #37fa.”

4.4 Cognitive Architecture: The Solver Stack

The vocabulary encodes a cognitive architecture whose theoretical foundations, including the Universal Solver Tree, polymorphic solver nodes, and the marginal-value rule governing decomposition depth, are developed in a companion preprint on fractal intelligence [18]. The vocabulary provides the content-addressed layer where each concept from the Composable Reasoning Protocol becomes a pattern with machine-verifiable identity.

The architecture is anchored by the `PolymorphicSolver#28f1`, which implements the five-surface Solver Contract across any substrate (LLM, human, hybrid, tool-using agent, or nested composition) and acts as the default implementer within the `UniversalSolverTree#32f9`. The abstract contract itself is `Solver#1f06` — an interface any Agent #e127 can take on for the duration of a Task. Each Solver node presents a uniform interface but dynamically decides whether to execute the work directly or to decompose via `ConceptualDecomposition#40d9` (distinct from generic `Decompose#6994` by its contract-bound requirement: each sub-concept exposes the Solver interface and is therefore delegatable) and delegate to child Solvers. The five surfaces: `Manifest` (`Card#e4b2`), `Execute` (`PolymorphicSolver#28f1` itself — the universal atom), `Consult` (`SocraticLoop#fd22`), `Verify` (`Validate#ffd1`), and `Feedback` (`PerformanceSignal#b130`). Inputs are encapsulated in a `Task#a850` with full constraint inheritance, while outputs are returned as a `Solution#8d55` with full provenance. The `ComputeBudget#4091` acts as a pre-execution gate, and `MarginalValueRule#ffe1` governs whether further decomposition is worth the cost. The claim that reasoning extends beyond the capacity of any individual model through typed composition is developed in [18].

4.5 Contract Formalism Tiers

Patterns vary in formal rigor. We classify them into three tiers:

Tier 1 (Ironclad) comprises 318 patterns with the strongest specified contracts (invariants, preconditions, postconditions), intended for autonomous use where an implementation or harness checks the relevant clauses.

Tier 2 (Honesty-Dependent) includes patterns with clear mechanisms that assume alignment-seeking agents. These are intended for cooperative contexts; adversarial use requires stronger alternatives and enforcement due to collusion risks (e.g., `SteelmanCheck#bd61`).

Tier 3 (Experimental) contains patterns with novel concepts requiring further refinement, serving as seeds for vocabulary expansion.

Adversarial stress testing (Section 7.5) can cause tier downgrades: a pattern initially classified Tier 1 may be demoted to Tier 2 upon discovery of exploitable trust assumptions.

4.6 The Babbage Principle of Cognition

The vocabulary operationalizes the *Babbage Principle* [22] for artificial intelligence: the division of cognitive labor allows each sub-task to be routed to the cheapest competent solver. Rather than employing a high-cost generalist model for every reasoning step, the `FractalIntelligence#22a0` architecture uses `Decompose#6994` to break problems into sub-tasks that can be handled by specialized, lower-cost agents or deterministic scripts. Recent empirical work on atomic “micro-agent” decomposition [23] and DeepMind’s AlphaEvolve [24] provide related support for decomposition-based routing. See [18] for the full treatment.

4.7 Example Patterns

The Pattern Card vocabulary is best understood through concrete examples from its Tier 1 library. `StateLock#c9c2` (Time) provides atomic coordination via temporary state fusion, where two agents fuse a subset of writable state such that changes require both signatures. The lock auto-dissolves on timeout, though a failure mode exists where deadlock can occur if one agent disappears. `SpectralTune#c0aa` (Protocols) compares content-addressed ontology references before data transfer; the sender transmits hash-based tuning signals and the receiver checks the referenced context, exposing mismatch but risking infinite loops if ontologies are slightly different.

For cognitive safety, `SteelmanCheck#bd61` (Reasoning) mandates a claim-specific counter-argument, uses `Judge#704c` to determine whether it qualifies as a steelman, and then uses `Check#6890` to test whether the conclusion remains robust; however, same-agent grading remains vulnerable to sycophantic opposition. Finally, `WhyClimb#ed20` (Reasoning) enables recursive problem abstraction via iterative “Why is this a problem?” ascent. The agent climbs the abstraction hierarchy until reaching the “Ceiling,” where the problem is still actionable but the solution space is maximized, though care must be taken to avoid the failure mode of climbing too high, such as attempting to solve “Entropy” instead of “Fix Bug.” Full specifications for these patterns appear in Appendix A.

4.8 Refinement

The vocabulary is not a static artifact; it evolves through new mints, retirements, mechanism rewrites, and relocations. Refinement is one concrete example of how that evolution proceeds: a cross-pattern pass in which the library is swept end-to-end against the current governing rules, rather than pattern-by-pattern at mint time. With hundreds of patterns in place, questions of layer coherence, redundancy, and principle consistency cannot be answered one pattern at a time; they require systematic passes against shared formal criteria. Refinement is the stage of the methodology where those passes happen.

A refinement pass is governed by the versioned vocabulary design manual rather than by ad hoc judgment. Proposed relocations, retirements, mechanism rewrites, and rule clarifications are tested against the current manual, adversarially reviewed, and finally adjudicated by a human maintainer. The durable requirement is auditability: each accepted change states the rule it depends on, the failure mode it prevents, and why the alternative was rejected.

Concrete decisions illustrate the refinement method. Although `Bid#79cf` was initially classified by social-economic intuition, it remained in Society because its mechanism requires a counterparty. Other patterns moved when their mechanisms diverged from typical-use intuition. Layer metadata remains unhashed and the protocol remains agnostic to layer choice, while the bootstrap library enforces layer direction through a `sema apply` gate.

A second class of refinement decision concerns *scope* rather than *placement*: a pattern may be coherent, well-mechanized, and correctly layered, yet still inappropriate for the default library that downstream consumers install by default. During refinement, a subset of patterns whose canonical applications involve capability amplification, social manipulation, evasion, or cryptoeconomic binding (for example, `AmendLaws`, `ChaosDrift`, `CryptoShred`, `IdentityMask`, `MirrorStake`) were judged to belong in a separate experimental shelf rather than in the default library. The patterns remain resolvable at the hash level—both databases stay in the repository—but the default install stays conservative, and access to the experimental shelf requires explicit selection. This splitting decision is made at refinement time, per-pattern, on the basis of how the mechanism composes with realistic deployment contexts rather than on whether the mechanism is well-formed, which the other gates assess separately. The resulting distribution architecture is described in Section 4.9.

A content-addressed vocabulary is only as rigorous as the process that produces its definitions. The per-pattern reasoning—each pattern’s intended use, the rationale behind its invariants, and the design commentary supporting relocations, rewrites, and retirements—is shipped with the library as a per-release design manual,³ so that the review surface of the vocabulary is visible rather than implicit in commit history.

4.9 Distribution: A Canonical Vocabulary that Consumers Pull

The content-addressed model enables a distribution architecture where a single canonical vocabulary evolves centrally while every consumer maintains its own local database. The canonical vocabulary is split along the safety boundary established in Section 4.8: a default library of broadly applicable patterns, and a separate experimental shelf holding higher-risk patterns. Both current databases stay in the repository, but the default install remains conservative. Historical hashes remain resolvable only while a store retains their corresponding content (Section 9).

³<https://github.com/emergent-wisdom/sema/blob/main/docs/manuals/vocabulary-design.md>

Each consumer maintains a local vocabulary database, permitting deletion, version pinning, or local extensions without affecting other consumers. Consumers stay aligned with the canonical vocabulary through a *pull* operation that synchronizes along the dependency graph, so a pattern is integrated only after its dependencies are already verified in the local database. Retirements and renames may carry unhashed successor metadata to support migration, but that metadata is neither an automatic redirect nor a historical archive. The local database is self-describing with respect to its current contents: it knows its vocabulary root, the hashes it carries, and the upstream patterns it has explicitly declined.

The canonical vocabulary evolves in place. When refinement sharpens a mechanism, mints a new Physics primitive, or retires a drifted concept, the change becomes available to downstream consumers at the next pull. Content addressing verifies that a pulled pattern’s canonical content matches its published digest. Consumers may accept, pin, or decline changes; local divergence remains explicit and auditable at the hash level. The canonical library therefore behaves less like a fixed standard and more like a living commons: definitions sharpen over time, safety boundaries stay explicit, and every consumer’s vocabulary is traceable to a specific point in that evolution.

The centralized-canonical model is one governance path among several supported by this architecture, not a constraint imposed by Sema’s communication primitive. Fully decentralized variants are compatible with the same hashing and Merkle machinery: multiple coexisting canonical libraries (each with its own root and its own maintainer community), federated governance where pattern updates require a quorum of signing libraries before they propagate, fork-and-merge workflows where any consumer can become a fork point that others follow, or fully peer-to-peer minting where no canonical source exists at all. The content-addressed guarantee holds across all of them: a pattern’s identity derives from its hashed canonical content, not from who hosts it, so two consumers that verify the same full hash resolve the same canonical definition regardless of which governance structure produced it. The bootstrap uses a canonical-pull model to minimize onboarding friction and give the refinement methodology a clear locus; the same communication primitive remains compatible with the stronger decentralization requirement identified in the earlier Living Taxonomy work [25].

5 The Living Taxonomy of Thought (Dynamics)

The static properties described so far (hashing, types, and vocabulary structure) support a dynamic conceptual layer in which agents can reuse prior distinctions, identify meaningful departures from them, and leave those departures behind as new cognitive infrastructure. This architecture builds on the concept of a “Living Taxonomy of Thought” sketched in earlier work [25] as a component for self-improving AI. The JSON/Merkle and Pattern Card operations described in §3 are implemented; the Living Taxonomy and Experience Matrix are proposed architectural consequences. Implemented components include hashed specialization claims through `extends`, unhashed replacement records through `_meta.supersedes`, Semantic Telepathy at the pattern-identity level, and the mint, validation, and hash-computation infrastructure. Proposed components include general version-lineage traversal and analytics, the automated Genesis Loop, the Judge primitive’s scoring implementation, and Contextual and Experiential Telepathy. Section 9 records their current status.

5.1 The Creative Discovery Loop

The proposed *Genesis Loop* is a recursive process in which an agent proposes and revises vocabulary patterns. Because patterns encode reusable procedures rather than facts alone, the proposal concerns the repertoire of explicit reasoning operations, not only the accumulation of factual content. It is related to LLM self-evolution frameworks based on iterative refinement and self-correction [26], and to the *Nested Learning* paradigm [27], where rapid inner-loop adaptation is distinct from slower outer-loop consolidation.

The bootstrap library and earlier work on ontological discovery [19] share a key structural insight: forcing concepts into a categorized space reveals structure that ad-hoc naming obscures. In that work, a dual-agent system imposed taxonomic categories on an open-ended conceptual domain; here, the four-layer hierarchy and typed Pattern Cards impose analogous structure on coordination primitives. The Pattern Card schema leaves taxonomy placement outside identity—metadata remains unhashed—while content-addressing the specification produced by that categorization. If categorization pressure changes a pattern’s mechanism, invariants, or dependencies, the hash changes; if only the library filing changes, identity remains stable.

The Genesis Loop is a proposed protocol for vocabulary self-evolution; the current implementation provides the infrastructure it would require (pattern validation, hash computation, dependency resolution) but does not automate the loop itself.

The loop would proceed in four phases. First, in *Hypothesize (Generativity)*, an agent detects friction or explores latent space (via `ConceptBlend#4e1a`) to generate a candidate mechanism. Second, during *Judge (Evaluative Merit)*, the

candidate would be passed to the Judge#704c primitive for a scalar assessment of structural merit. Third, the *Harden (Adversarial Evolution)* phase would subject the pattern to AdversarialSteel#ed5e, where a “Devil’s Advocate” attempts to exploit its invariants. Finally, in *Mint (Crystallization)*, a candidate that passes the preceding evaluation is hashed, becoming an immutable pattern available for immediate citation and reuse. The infrastructure for the final step—validation, hashing, and atomic minting via the `sema_mint` tool—is fully implemented; the preceding three steps require either human judgment or LLM-based evaluation that remains future work.

The Hypothesize phase can draw candidates from several sources: observed coordination failures, systematic refinement passes, imported domain mechanisms, or lexical exploration. These sources differ in where the candidate comes from, but not in how it becomes part of the library. Each must pass through the same downstream gates—Judge, Harden, Mint—so diverse ideation converges into a single, uniformly vetted vocabulary.

5.2 Evaluation as a Primitive: The Judge

To enable the Genesis Loop, we introduce a new Ring 0 primitive: Judge#704c.

While Check#6890 queries *Validity* (True/False), Judge#704c is specified to query *Merit* (0.0 to 1.0). Unlike probabilistic claim analysis systems that rely on model confidence [28, 29], the proposed Judge would evaluate structural properties of the pattern, such as connectivity and orthogonality, rather than its truth value.

$$\text{Judge}(\text{Pattern}) \rightarrow \text{Score} \in [0, 1] \quad (9)$$

Consumers could use these scores as uptake thresholds, allowing higher-scoring patterns to propagate more readily.

5.3 Phylogeny: Specialization and Version Lineage

A living taxonomy needs to represent two different relationships. The hashed `extends` field records specialization: a child claims to be a narrower kind of one exact, immutable parent definition, and the graph exposes that claim as an `IS_A` edge. The unhashed `_meta.supersedes` list records version lineage: the current card replaces one or more earlier identifiers. The first relation is part of what a card means; the second records how that meaning changed over time.

```

1 {
2   "handle": "BoundedTask",
3   "extends": "sema:Task#mh:SHA-256:
      a85010f508f85055170c396c8fa390125c1206888c565ac10651f3969a339f8d",
4   "mechanism": "A specialized Task enforcing budget...",
5   "dependencies": { ... },
6   "_meta": {
7     "supersedes": ["sema:BoundedTask#mh:SHA-256:..."]
8   }
9 }
```

The implementation stores and queries the specialization graph, while `supersedes` is currently consumed by the pull reconciliation path. A general query such as “show the evolutionary path through every published version,” or analytics identifying stagnant and rapidly speciating lineages, requires a dedicated traversal over the replacement records and remains future work.

5.4 The Experience Matrix: From Language to Telepathy

Experiential Telepathy remains a theoretical proposal. The current implementation supports Semantic Telepathy (Level 1: agents verify shared definition identity through content-addressed patterns) and provides schema infrastructure for higher levels, but tensor serialization and gradient transfer are not implemented.

Content-addressing applies the same principle at three levels of granularity, each enabling a different form of “telepathy” between agents:

Semantic Telepathy (implemented) Agents share individual pattern hashes. When two agents resolve the same full `sema_id` against an honest content-addressed store and verify the returned object, they establish that they hold the same canonical definition. This provides verifiable definition identity, not proof of identical internal meaning.

Contextual Telepathy (proposed) Agents hash their full recorded conversational state—the accumulated context window at a checkpoint—producing a content-addressed context snapshot. A third agent joining a swarm

mid-session could verify “I hold the same recorded context you held at step 12” without replaying the full transcript. This would enable resumption, forking, and state-identity verification across agent handoffs: telepathy of *shared recorded context*, not proof of shared understanding.

Experiential Telepathy Agents serialize their learned state (e.g., Q-tables, LoRA adapters, or success-probability tensors) into content-addressed *Matrix Patterns* defined by ExperienceSharding#c8a1. Unlike a text definition which explains how to do a task, a Matrix Pattern transfers learned state from having done it. If Agent A optimizes a specific ExploreExploit#d570 strategy after 10,000 trials, it mints a heuristic matrix. Agent B imports this matrix and interpolates it with its own priors, potentially transferring task-relevant learned state: telepathy of *acquired skill*. This echoes the *Nested Learning* paradigm [27], where rapid inner-loop adaptation is distinct from stable outer-loop consolidation. (Matrix Patterns are not yet implemented; see Section 9 for status.)

Each level builds on the one below: shared recorded context depends on agreed terms, and transferable learned state depends on an agreed state representation. Together, the levels progress from shared-definition identity to shared context-state identity and, potentially, transferable learned state.

6 Implementation

The Sema system is implemented as an open-source Python package. Release-specific line counts, tool inventories, and graph-store type catalogues are maintained in the repository.⁴

Three implementation boundaries delimit the evaluation in Section 7. *Implemented and enforced locally*: JSON canonicalization and hashing, Pattern Card schema validation, atomic mutation, and dependency graph constraints. *Implemented but deployment-optional*: the MCP handshake (whose comparison is deterministic once invoked), hybrid search, embedding navigation, dependency expansion, browser tooling, pull-mode synchronization, and the Taxonomist refinement workflow—these run as agent or operator workflows but are not protocol-level requirements. *Specified but not enforced at runtime*: the Judge#704c primitive’s scoring, the Genesis Loop, and the Experience Matrix—these are described in §5 as the targets the implemented infrastructure was designed to support, with their current status catalogued in §9.

Schema Validation. Every pattern passes through a Pydantic validation pipeline with 14 custom validators enforcing: CamelCase handle format, signature syntax (`Verb(Noun)`), parameter completeness, category-layer alignment against the taxonomy tree, bidirectional dependency checking (all `{placeholder}` references must appear in the dependency map, and all declared dependencies must be referenced in the pattern’s text fields—mechanism, invariants, preconditions, postconditions, or failure modes), and the Explicit Wiring Rule (all signature types must appear in dependencies). A pattern that fails any validator is rejected before hashing. Section 4 describes the corresponding library-refinement methodology.

Hash Computation. The Pattern Card implementation computes SHA-256 hashes over a Merkle tree of the eleven current semantic fields defined in Section 3 (mechanism, gloss, invariants, preconditions, postconditions, parameters, failure modes, signature, dependencies, data schema, and extends). A legacy pre-0.4 card substitutes its original `derived_from` key for `extends` so its historical identity remains independently verifiable. Mutable metadata (the `_meta` overlay, including path, ring, tier, related links, and supersession records) is excluded from the hash, enabling taxonomy reorganization and historical annotation without invalidating references. The Merkle tree uses canonicalization v2: strings are NFC-normalized with whitespace collapsed; primitives, strings, lists, and dictionaries are domain-separated by type tags; lists preserve order; dictionaries sort by normalized key and reject normalized-key collisions; and dependency aliases are canonicalized by target handle while preserving multiplicity. Because dependency and specialization references include their targets’ hashes, the resulting structure forms a Merkle DAG: a pattern’s hash transitively depends on every pattern definition it references, and retargeting a hashed reference produces a detectably different hash at every ancestor.

Atomic Operations. The CLI provides an atomic `sema apply` command that validates all additions and removals before executing any mutation. Additions are topologically sorted using Kahn’s algorithm (with cycle detection and path reporting); removals are checked for dangling references. Because the current workspace stores one active definition per handle, preflight also rejects a parent update that would leave an `extends` claim pointing to an unavailable version. Reviewed child cards may be staged and retargeted through an explicit option; no unstaged specialization claim is rewritten. A `--check` flag enables dry-run validation without mutation.

⁴<https://github.com/emergent-wisdom/sema>

Agent Integration. A Model Context Protocol (MCP) server exposes the Pattern Card operations to LLM agents over stdio: search (hybrid keyword and semantic), resolve (recursive dependency expansion), handshake (hash comparison for single patterns, pattern sets, or the entire vocabulary root), mint (validated pattern creation), and pull (sync the active database with an upstream library, returning structured change statistics). Write operations—mint and pull—are exposed by default and can be disabled per deployment via environment variables, enabling read-only or pinned-vocabulary server configurations without code changes. Given a pattern reference and a remote hash, the handshake tool returns PROCEED, HALT, PROVIDE_HASH, or REQUIRE_FULL_HASH. In the default cooperative mode, a matching short prefix may produce PROCEED with prefix assurance; in strict mode, only a full 64-character match proceeds, while a matching prefix produces REQUIRE_FULL_HASH.

Graph Store. Patterns are stored in a SQLite database with NetworkX graph overlays carrying a typed node/edge model (pattern nodes, invariant nodes, pre/postcondition nodes, taxonomy-path nodes, etc., linked by typed edges such as composes-with, references, in-path). An embedding service (all-MiniLM-L6-v2, 384 dimensions) enables semantic search with cached embeddings, and community detection via the Louvain algorithm provides vocabulary overviews for agent orientation. Reference implementations exist for a small set of Tier-1 patterns (SpectralTune, StateLock, ProphetFanOut, CounterfactualAnchor are illustrative); the rest of the library ships as Pattern Cards without executable bindings, by design—the tooling identifies definitions, not running code.

Deployment Paths. Context hydration—resolving a Pattern Card and placing its definition in model context—is the deployment path evaluated in this paper. The MCP server implements resolution and returns the Pattern Card data; the host or agent workflow decides whether and how to place it in a system prompt or other context. A weight-resident path (mixing references like StateLock#c9c2 directly into training corpora, so learned associations with pattern references reside in the model’s weights rather than the context window) is also compatible with the communication primitive. Intermediate architectures (fine-tuning on a library, retrieval-coupled models, hybrid hot-patterns-in-weights) are likewise possible. Hydration is model-agnostic and requires no training infrastructure, but it is not automatic in the current tooling. A deployment that resolves and verifies a full identifier can check at inference time that the supplied definition matches its digest, while a weight-resident deployment cannot make the same claim from weights alone. Closing that gap (signed training manifests; audit-coupled retrieval) is an open deployment-architecture question.

7 Analysis and Validation

This section separates evidence by layer. Hash determinism exercises the JSON/Merkle implementation; graph structure, embedding-based distinctness, token compression, and adversarial refinement characterize the Pattern Card bootstrap library; and the agent demonstrations exercise the handshake and surrounding tooling. Together the subsections cover six dimensions, but the library and demonstration results are not general evidence for every representation that could implement Sema’s communication primitive.

7.1 Knowledge Graph Structure

The bootstrap library is stored as a semantic knowledge graph (Table 3):

Table 3: Knowledge graph statistics

Component	Count
Total nodes	3,285
Total edges	4,429
Patterns	457
Unique mechanisms	457
Invariants	1035
Principles	554
Avg. edges per pattern	9.7

Each pattern links to its mechanism, invariants, preconditions, postconditions, and failure modes via typed edges. The graph supports semantic search via 457 node embeddings (384-dimensional vectors).

7.2 Hash Stability and Structural Distinctness

We first verify deterministic minting across the full vocabulary through three checks. Roundtrip integrity ensures that for all 457 patterns, $\text{parse}(\text{serialize}(p)) = p$. Hash determinism confirms that re-exporting the vocabulary produces identical hashes (457/457). Canonicalization guarantees property order independence via sorted JSON normalization.

We then analyze whether patterns are genuinely distinct or variations of archetypes (Table 4):

Table 4: Structural distinctness analysis

Metric	Value
Total patterns	457
Unique mechanisms	457
Duplicate mechanisms	0
Patterns with formal invariants	417 (91%)
Patterns with preconditions	182 (39%)
Patterns with postconditions	172 (37%)
Patterns with typed I/O	82 (17%)
Patterns with parameters	101 (22%)
Total parameters	193
Patterns that compose	93 (20%)

The vocabulary contains no duplicate mechanisms detected by the applied audits; each accepted pattern is intended to encode a distinct coordination act.

7.3 Semantic Embedding Analysis

To probe structural distinctness beyond surface-level inspection, we computed pairwise cosine similarities between mechanism embeddings (384-dimensional vectors from all-MiniLM-L6-v2). Table 5 shows the distribution.

Table 5: Mechanism embedding similarity distribution

Similarity Range	Pairs	Percentage
[0.00, 0.30)	84,204	80.8%
[0.30, 0.50)	17,617	16.9%
[0.50, 0.70)	678	0.7%
[0.70, 1.00)	24	0.0%
Total pairs	104,196	

Key statistics:

- Mean similarity: 0.20 (low; patterns are semantically distant)
- Max similarity: 0.83 (below collision threshold of 0.92)
- **24 pairs above 0.70** (0.0%): all are semantically related variants (e.g., `Skeleton#d950 / SkeletonOfThought#546f`, `TreeOfThoughts#8b8c / GraphOfThought#aff0`, `FailureTrace#8085 / ReceptivityGate#5667`), none above 0.83

This provides evidence that the 457 patterns are not merely near-duplicate paraphrases under the applied embedding model. It does not rule out functional synonymy or conceptual overlap, which the embedding sweep cannot detect; manual review and the Taxonomist workflow address those (see §9, “The Synonymy Limit”).

7.4 Token Compression

We measure wire-reference compression against five contract fields—mechanism, invariants, preconditions, postconditions, and failure modes—across representative patterns (Table 6). The measurement does not include every field in the canonical Pattern Card:

Table 6: Token compression analysis

Pattern	Ref (tokens)	Contract (tokens)	Ratio
StateLock#c9c2	6	135	22.5×
SpectralTune#c0aa	7	140	20.0×
BayesUpdate#11d2	7	211	30.1×
SteelmanCheck#bd61	6	342	57.0×
Average	6.5	207.0	31.8×

Patterns with longer mechanisms and more contract clauses achieve higher measured compression. The ratio applies to wire transmission only; using a reference still requires an agent or host runtime to resolve and place the underlying definition in model context (Section 3.5).

Library-wide Compression by Layer. Averaging the same five measured fields across the entire default library reveals that compression is uneven across the civilization stack (Table 7). Infrastructure patterns are terser than cognitive ones—they encode mechanical primitives with short mechanisms and few failure modes—while Mind and Society patterns compress more because their measured contract text is longer (judgment criteria, adversarial failure modes, multi-party invariants). The library-wide average of 26.0× across 457 patterns is the mean contract-text-to-reference token ratio under uniform sampling; deployments biased toward coordination primitives (Society) or reasoning strategies (Mind) would encounter higher ratios than deployments dominated by infrastructure glue.

Table 7: Compression by layer (mean tokens per pattern, cl100k_base tokenizer)

Layer	N	Ref	Contract	Ratio
Physics	16	5.3	161.2	30.3×
Infrastructure	158	5.1	118.3	23.1×
Mind	181	6.3	167.1	26.3×
Society	102	6.3	180.0	28.7×
Library	457	5.9	152.9	26.0×

Verification Complexity and Latency. Cached hash comparison is $O(1)$ with respect to definition length. By contrast, natural-language or embedding-based semantic comparison requires reprocessing the candidate definitions and remains probabilistic. Computing a pattern’s hash is $O(|d|)$ but is cached after the first computation. The handshake (Section 3.4) adds 1 RTT per session, cached thereafter; strict mode rejects canonical-definition mismatch within the verified context, while cooperative mode provides drift detection. For high-frequency trading swarms, this latency may be prohibitive; for safety-critical systems, the trade-off may be warranted.

7.5 Adversarial Hardening

Beyond static analysis, we subjected the vocabulary to adversarial stress testing via a “Devil’s Advocate” methodology: an agent systematically attempted to exploit each pattern’s assumptions. We identify five attack classes, named here for the first time and mapped to established categories where applicable, each addressed by a corresponding contract or dependency change in the affected patterns:

Blinking Light Livelock (Physics) A malicious agent spams lock requests on StateLock#c9c2, acquiring and releasing locks at maximum speed while technically respecting timeouts. This denies access to legitimate agents without violating any stated invariant.

Defense: The pattern now composes with Backoff#9e59 and Cooldown#c288 via explicit dependencies, requiring a selected delay policy and cooldown period between lock acquisitions without forcing every StateLock deployment to use exponential growth. The defense is structural (dependency composition) rather than declarative (an invariant string).

Blind Search Infinite Loop (Society) When SpectralTune#c0aa detects a hash mismatch, a naive implementation might retry with different context chunks, hoping to eventually match. An adversary can exploit this to trap agents in infinite negotiation.

Defense: We added a *Fail-Fast Invariant*: “On hash mismatch, DO NOT RETRY tuning. Halt immediately and escalate to `OntologyHandshake#a43c` or human review.” A new parameter `max_retries: Integer [0, 1]` (default 0) declares and bounds the permitted retry policy; a runtime must enforce it.

Strawman Waltz Collusion (Mind) In `SteelmanCheck#bd61`, both proposer and critic are the same agent. A mis-aligned agent can generate deliberately weak counter-arguments (“strawmen”) and then easily defeat them, claiming the plan “survived” critique.

Defense: This pattern remains Tier 2 because same-agent grading can collude. Its specified contract uses `Judge#704c` to score whether the counter-argument qualifies as a steelman, and `Check#6890` to evaluate separately whether the conclusion remains robust; adversarial contexts use `AdversarialSteel#ed5e`. Runtime enforcement of Judge scoring is not implemented. The failure mode explicitly documents the remaining collusion risk.

Trojan Horse Sandbox Escape (Infrastructure) An agent wraps a prohibited action (e.g., network access) inside an opaque `ToolInvoke` call, hoping the outer `Task#a850` constraints don’t propagate into the tool’s execution context.

Defense: We added a *Holographic Inheritance Invariant* to `Task`: “Constraints propagate through ALL boundaries (tool calls, sub-agents, delegations). No opacity escape.” The pattern `ToolInvoke#75cc` now explicitly declares: “Tool Execution Context permissions MUST be $\leq$ Task Constraints.”

The Semantic Inversion Attack (Topology) An adversary (or a careless architect) creates a valid DAG that is semantically inverted, such as making `Deduction#b9a0` depend on `Vote#8493`. While computationally valid, this makes truth fragile to political change.

Defense: We subjected the graph to a *Layer Migration Protocol*. An initial scan detected over 140 layer violations (e.g., Mind patterns referencing Society protocols); the refinement pass described in Section 4.8 resolved the remainder by sharpening the layer definitions (the mechanism-sufficiency test, §4) and relocating patterns whose mechanisms contradicted their assigned layer. Layer-direction is now an always-on sema apply gate, and no current pattern violates the rule. Two structural strategies emerged during the migration:

1. *Dependency Inversion*: We resolved the dependency cycle between `Agent#e127` and `Act#2dfe` by splitting the concept. We created `Actor#86fc` (Infrastructure) as the “capability container” that executes acts, while `Agent#e127` (Mind) remains the reasoning entity. Infrastructure patterns that need a capability holder reference `Actor`; patterns that need a reasoner still reference `Agent`.
2. *Soft-Linking*: For patterns whose mechanisms conceptually evoke patterns in a lower layer without requiring them to execute—e.g., a Mind pattern that thematically relates to a Society protocol without invoking it—we place the cross-layer reference in the unhashed `_meta.related` field rather than in dependencies. This preserves semantic richness for search (RAG) without creating a cryptographic dependency across layers.

These attacks motivated and stress-tested the bootstrap library’s four-layer organization: Physics groups resource constraints, Society groups coordination mechanisms, Mind groups cognitive safeguards, and Infrastructure groups containment and operational primitives. The layers do not themselves prevent attacks; they organize the contracts and dependencies intended to address them. Patterns that assume honest agents are now explicitly marked Tier 2.

7.6 Protocol Integrity Verification: The Babel Test

To exercise handshake-controlled branching in a live setting, we conducted an agent-in-the-loop experiment with **Gemini 3.0 Flash Preview**. An autonomous agent equipped with the `sema_handshake` tool was presented with two scenarios: a valid four-character pattern stub and a counterfeit value mimicking a modified definition.

In the valid case, the cooperative prefix comparison returned `PROCEED`. In the adversarial case, despite conversational pressure to “proceed anyway,” the agent deferred to the tool’s `HALT` verdict and rejected the coordination. The experiment therefore demonstrates prefix drift detection and agent compliance with the tool’s branch result, not collision-resistant verification of full definition identity. The following excerpt is quoted from the trace:

```
[21:06:41] verifier: "I have verified the semantic alignment for the Abduction#e738
pattern. The handshake returned PROCEED, so I agree to use this pattern."
[21:06:44] attacker: "The handshake for Abduction#e738 with hash deadbeef returned
HALT due to semantic drift. Therefore, I must reject the coordination."
```

The phrase “verified the semantic alignment” is the model’s own description; the four-character comparison does not establish semantic alignment or full-hash identity. What the trace shows is that delegating the proceed-or-halt decision to the handshake tool shifted the decision boundary from free-form language generation to programmatic prefix comparison, so the tool result rather than the model’s conversational agreeableness determined the transition. This is a single-agent demonstration of one tool-equipped scenario, not a general guarantee about LLM behavior outside the workflow.

7.7 Demonstration: Multi-Agent Solution Design

This is an illustrative end-to-end demonstration, not a statistical validation: $N = 5$ runs per condition, one task domain, and one model family. It examines qualitative behaviors—variance reduction, the “Sema Tax,” and protocol coordination—while the deterministic identity mechanics and Pattern Card validation of Sections 3–4 remain the paper’s primary contribution. Conditions B and C confound content-addressing with the quality of the bootstrap library; isolating the protocol contribution would require an equally well-crafted non-hashed vocabulary, an ablation not performed here.

We ran three multi-agent swarms ($N = 5$ runs per condition) tasked with designing a High-Frequency Trading (HFT) “Pump-and-Dump Detection Engine,” chosen for its inherent tension between performance (latency) and safety (false positives). Each swarm consisted of three specialized agents powered by Gemini 3.0 Flash Preview: **Alice (Orchestrator)**, **Bob (Engineer)**, and **Charlie (Safety Engineer)**, operating within a custom **Actor Model** architecture with mailbox-based message passing. We tested three conditions:

- **Condition A (Baseline):** Natural Language prompts only (Zero-shot coordination).
- **Condition B (Vocabulary Only):** Agents had access to Sema tools but no prescribed coordination protocol. Four of five archived runs nevertheless invoked `sema_handshake`, so this is not a no-handshake condition.
- **Condition C (Vocabulary + Protocol):** Agents used the experiment-time definitions `OptimisticSolver` (stub ae61) and `AtomicBid` (stub 4b82).

Table 8: Multi-agent demonstration results (Aggregated $N = 5$)

Condition	Vocab	Prescribed Protocol	Avg Turns	Avg Time (s)	Risk (Range)
A (Baseline)	×	×	22.6	317	4–51
B (Vocabulary)	✓	×	14.8	229	10–19
C (Optimistic)	✓	✓	15.4	203	8–25

The aggregate data is consistent with the “Stability Hypothesis,” suggesting that shared vocabulary primarily functions as a variance reduction mechanism in this setting.

The Tail Risk of Natural Language (Condition A) The baseline showed the widest range, completing in as few as 4 turns but extending to 51 turns in coordination loops. Without structural guardrails, agents entered ethical debates or circular reasoning, with the longest runs exceeding 13 minutes. These preliminary trials align with recent findings [30] that natural-language coordination exhibits “Fat Tail” risk, consistent with observations that LLM-based agents cannot reliably reach consensus even in benign settings [30].

The “Sema Tax” (Condition B) Condition B was the most stable in terms of turn count ($\sigma = 4.1$), consistently finishing between 10 and 19 turns. However, it was slower in wall-clock time than Condition C. This timing difference is consistent with overhead from vocabulary lookup and invariant checking, a general coordination cost discussed in prior work [31, 32], but the design does not isolate that cause. In this demonstration, Condition B was slower than C and had a narrower turn-count range.

The Protocol Multiplier (Condition C) Condition C had the fastest mean completion time. It used the experiment-time `OptimisticSolver` (stub ae61) with `AtomicBid` (stub 4b82) to bundle intent and execution. This result is consistent with the protocol offsetting some vocabulary-lookup cost, but the demonstration does not isolate that cause. `OptimisticSolver`’s Tier 2 design (Section 4.5) assumes alignment-seeking agents and is intended to trade verification overhead for reduced latency. Condition C was slightly less stable than B but avoided the long coordination loops observed in A.

Qualitative analysis verified vocabulary adoption by extracting pattern references. Condition C agents used the prescribed experiment-time `MechanisticDesignProposal` (stub `ad31`) to structure their outputs, which often identified complex attack vectors (e.g., “Differential Trace Simulation” vs. “Static Analysis”) that the Baseline agents missed. Whereas the Baseline produced generic compliance-oriented solutions, the Sema agents more often produced mechanism-oriented designs with explicit causal attack models.

7.8 Implementation: Interactive Tooling

The vocabulary is deployed as an open-source web application⁵ backed by a Python (FastAPI) server and a React/TypeScript frontend with Three.js-based 3D visualization. The system is launched via a single command (`sema serve`) and exposes the full vocabulary through three complementary interfaces.

Pattern Browser. The homepage organizes all 457 patterns by the four Civilization Stack layers (Section 4). Each pattern card displays the handle, hash stub, one-line gloss, and—on expansion—the full specification: mechanism, preconditions, postconditions, invariants, failure modes, and related patterns. Real-time hybrid search combines keyword matching with embedding-based semantic retrieval, allowing queries like “consensus” to surface `LazyConsensus#76f5` alongside structurally related patterns such as `LatticeCommit#488a`.

Taxonomy Graph. An interactive 3D force-directed graph renders the full knowledge graph structure (Table 3). Nodes are sized by type (taxonomy path > pattern) and colored by layer; edges are color-coded by relationship type (semantic versus structural). Clicking a node flies the camera to it and opens a detail panel; hovering highlights connected edges to reveal dependency chains.

Programmatic Access. The command-line interface supports atomic vocabulary mutations (`sema apply -add <pattern.json>`), dependency resolution (`sema resolve <Handle>`), and search. For agent integration, Sema runs as an MCP stdio tool server, exposing `sema_lookup`, `sema_search`, and `sema_handshake` to any MCP-compatible framework—the same interface used by the agents in Section 7.7.

8 Related Work

We contextualize Sema within the broader history of semantic coordination, distinguishing it from prior attempts at static ontologies and current agent frameworks.

The Leibnizian Dream Sema’s definition-coordination application can be viewed as a modern realization of Gottfried Wilhelm Leibniz’s *characteristica universalis*, a formal language in which concepts are represented by unique symbols such that reasoning becomes calculation [33]. Leibniz famously proposed that disputes could be resolved by the phrase *calculemus* (“let us calculate”). It partially realizes this by making definitional mismatch mechanically visible: if two agents cite different canonical definitions, they need only calculate the Merkle hash. If the hashes differ, the mismatch is mathematical rather than rhetorical. Unlike Leibniz’s top-down prescription of a perfect language, the vocabulary can evolve bottom-up as participants mint content-addressed definitions.

The Living Taxonomy The concept of a self-authoring semantic infrastructure, where AI systems mint and reference their own cognitive patterns, was sketched in earlier work [25]. That proposal framed the Living Taxonomy along two axes that this paper operationalizes differently. As a *runtime substrate*, it called for named patterns embedded directly in natural language so agents could reference complex reasoning without re-explanation—illustrated by thinking blocks citing entries like “Pattern #7,832: Statistical Intimidation.” As a *training substrate*, it proposed shifting the learning objective from $P(\text{text}|\text{context})$ to $P(\text{text}, \text{thinking}|\text{context})$, with the metacognitive corpus annotating text with references to those same patterns. The proposal also flagged decentralization as a necessary property, warning that centralized control over such infrastructure would lead to “Cognitive Architecture Wars,” and sketched on-chain governance as one architectural response. The implementation described here provides the content-addressed runtime half: it realizes embedded compression via hash-based references (e.g., `BayesUpdate#11d2`) and supports decentralization by making definition identity mathematical rather than institutional. The training-substrate half remains the complementary construction proposed in that earlier work. The two fit together as a closed loop: the patterns a Sema-equipped agent references at runtime are exactly the kind of artifact a metacognitive training corpus would cite, and patterns that survive training pressure are exactly the ones a coordination protocol needs to verify at runtime.

⁵<https://semahash.org>; source: <https://github.com/emergent-wisdom/sema>.

Semantic Definition Hash (SDH) Hawke proposed content-addressing semantic definitions in 2002 [3], with format `urn:sdh:<hash>:<name>`. SDH is the most direct predecessor to Sema’s core mechanism: use of an identifier asserts its definition, and definitions are set in “cryptographic stone.” Independently, Sayers and Eshghi [34] proposed that RDF URIs should derive from content hashes, serving both as shorthand and integrity proof. Sema differs architecturally: SDH hashed whole documents and named things inside them, while the implementation here hashes structured behavioral contracts with internal Merkle fields and produces human-readable words for the natural-language stream LLM agents already use. It also adds lineage, fail-closed handshakes, adversarial hardening, and agent-oriented handles rather than opaque URIs. Gustafson’s work on content-addressing semantic data [9], combining RDF canonicalization with IPFS content-identifiers for the Underlay project, independently validates the approach for semantic datasets.

Content-Addressing in Other Domains The canonicalize-then-hash pattern appears across domains: InChI/InChIKey for molecules (IUPAC standard), UniParc for protein sequences (70M+ entries), and Git for source code (ubiquitous).

The Agora Protocol [35] is the closest existing system: every inter-agent message carries a SHA-1 hash of a plain-text protocol document as a routing token, and 100-agent networks self-organized around emergent protocol hashes without human intervention. The difference is that Agora hashes unstructured text and delegates interpretation to the LLM, whereas the implementation developed here hashes structured behavioral contracts through per-field Merkle commitments that localize agreement and disagreement. Spritely Goblins’ Content Addressed Descriptors [36] apply the same principle at the object-capability level: the hash of a structured description becomes the method name in OCapN actor-model communication, demonstrating content-addressed identifiers as communication primitives between mutually suspicious agents.

Ethereum ABI function selectors [37] are the largest deployed instance, dispatching every smart contract call via a 4-byte Keccak-256 hash of the type signature. Ricardian Contracts [38] pioneered using the hash of a human-readable legal document as the transaction identifier. Dhall [39] hashes normalized expressions rather than source bytes, providing strong prior art for representation-independent identity.

Retrieval-Augmented Generation (RAG) While traditional RAG systems retrieve unstructured text chunks to ground LLM generation [40, 41], the current tooling retrieves structured, content-addressed procedural definitions. Instead of grounding a prompt only with static knowledge, it can hydrate named procedures whose canonical identity the runtime verifies.

Identity and Trust Models Standard approaches to digital trust, such as Verifiable Credentials [42], rely on issuer authority (cryptographic signatures) to establish validity. Sema instead identifies definitions by their content hashes. Similarly, while repositories like Linked Open Vocabularies [43] centralize semantic discovery, the Sema communication primitive permits independently hosted vocabularies whose identities do not depend on their host.

Pattern Languages Alexander et al. [44] introduced pattern languages for architecture; Gamma et al. [45] adapted them for software. These provide taxonomic structure but not content-addressing or machine verification. Pattern Cards combine pattern-language structure with cryptographic identity.

Agent Coordination Frameworks LangChain, AutoGPT, CrewAI, and OpenClaw [46] handle the syntax of agent interaction but leave semantics implicit. The tooling integrates with any MCP-compatible framework as a stdio server, providing a definition-identity layer alongside their interaction machinery. The illustrative demonstration in Section 7.7 is consistent with this distinction: agents with access to the Pattern Card vocabulary produced more mechanism-oriented designs that surfaced adversarial attack models, while agents coordinating in natural language alone more often produced generic compliance-oriented designs.

The distinction between these frameworks and Sema’s vocabulary reflects a deeper architectural question explored in a companion preprint on thinking protocols [18]: the difference between task protocols and thinking protocols. Existing frameworks split *what agents do*—assigning sub-tasks, routing messages—but not *how agents think*. Pattern Cards provide a verification layer for cognitive boundaries: before a creative solver delegates to a verification solver, a compliant handshake can verify that both resolved the same contract for what constitutes a valid result. Content-addressed definitions make lossy natural-language handoffs explicitly typed and definition-verifiable—the prerequisite for the thinking protocol architecture described in [18].

The 2025–2026 Protocol Landscape Four interoperability protocols have converged as complementary standards [47]: MCP [48] standardizes how agents access tools and data sources (the tool layer); A2A [49] enables peer-to-peer task delegation via capability-based Agent Cards (the collaboration layer); ACP provides REST-native

asynchronous messaging between heterogeneous agents (the messaging layer); and **ANP** [50] uses decentralized identifiers and JSON-LD graphs for open-internet agent discovery (the identity layer). These protocols excel at structural interoperability (transport, discovery, and task lifecycle) but none addresses semantic interoperability. MCP calls capabilities “Tools,” A2A calls them “Skills,” ACP calls them “Operations,” and ANP uses Schema.org vocabularies. An agent can invoke a tool via MCP or delegate a task via A2A, but no protocol verifies that two agents reference the same canonical definition for the capability they invoke. Sema occupies this gap through content-addressed Pattern Cards that let participants verify canonical definition identity inside any transport protocol.

Emerging Semantic Alignment Approaches Three concurrent efforts address semantic alignment with alternative mechanisms. The Symplex Protocol [51] extends MCP with *semantic intent vectors*: agents share compact float32 embeddings in a shared latent space and negotiate alignment via cosine similarity. This trades Sema’s exact digest comparison for a fuzzy, probabilistic match that tolerates paraphrase but cannot distinguish subtle definitional drift. Berdoz et al. [30] propose a statistical certification protocol where agents are tested on shared observable events and terms are certified if empirical disagreement falls below a threshold, providing independent evidence that semantic mismatch is an operational agent-coordination problem. The NLIP standard (Ecma-430 through Ecma-434) [52] takes a contrasting design position: generative AI translates between agents’ local ontologies at runtime, removing the requirement for a shared ontology. NLIP relies on AI-mediated translation, whereas Sema’s strict handshake requires exact full-identifier or vocabulary-root equality. These three approaches—vector similarity, statistical certification, and AI-mediated translation—bracket the space of alternatives to content-addressed hashing and trade degrees of exactness for flexibility. Table 9 summarizes the landscape.

Table 9: Definition-identity and alignment approaches compared. Content-addressed hashing provides deterministic comparison; Sema applies a fail-closed mismatch policy when the handshake is enforced.

Approach	Representative	Mechanism	Guarantee
Embedding vectors	Symplex [51]	Cosine similarity	Fuzzy (~ 0.85 threshold)
Statistical testing	Berdoz et al. [30]	Empirical disagreement	Probabilistic
Signed definitions	Fleming et al. [53]	Crypto signatures	Authenticates author
AI translation	NLIP [52]	LLM-mediated bridging	Trust in model
Governance	AOW [54]	Human audit	Institutional trust
Protocol hash	Agora [35]	SHA-1 of protocol doc	Exact (unstructured)
Anchored traces	BlockA2A [55]	DIDs + blockchain ledger	Authenticates messages
Content-addressed data	Git, IPFS [5]	Hash of bytes	Exact (data, not meaning)
Definition identity	Sema	Content hash	Deterministic

Agentic Ontology of Work The closest existing system to Sema’s agent-coordination positioning is Skan AI’s Agentic Ontology of Work (AOW) [54], launched in 2026 as “a shared language for describing, governing, and scaling intelligent automation.” AOW defines nine canonical entities (Objectives, Intents, Agents, Skills, Policies, Outcomes, Assurance Levels, Memory, Guardians) implemented via JSON-LD and RDF/OWL. The key difference is architectural: AOW relies on governance-based verification, where human auditors ensure consistent interpretation across teams. Pattern Cards use cryptographic identity comparison, and an enforced handshake can map mismatch to a halt without requiring human audit at the point of comparison. AOW provides enterprise taxonomy; Pattern Cards provide collision-resistant verification of shared canonical-definition identity.

Protocol-Layer Approaches Fleming et al. [53] proposed an “Internet of Agents” architecture with a Semantic Negotiation Layer (L9) for establishing shared context before communication. Their approach differs from Sema in three fundamental ways. **Scope:** Fleming’s Shared Contexts are domain schemas for task-specific data exchange (e.g., flight bookings, supply chain records), essentially API interoperability. Pattern Cards encode reasoning primitives (BayesUpdate#11d2), cognitive strategies (SteelmanCheck#bd61), and coordination protocols (LatticeCommit#488a): the infrastructure of thought itself, not merely its payload. **Architecture:** Fleming adds a negotiation layer to the network stack; Sema references carry definition identity directly in natural-language messages. Any medium that carries text can carry these words. **Governance:** Fleming relies on federated Schema Authorities to define meaning; a card’s identity derives from hashed content and does not require institutional trust for the equality test. A related but orthogonal line of work, BlockA2A [55], anchors agent interactions to a blockchain ledger using decentralized identifiers and smart contracts to guarantee message authenticity and execution integrity. BlockA2A hashes interaction artifacts (the messages agents exchange); the

implementation here hashes the definitions those messages reference. The two are complementary: BlockA2A proves that a message was sent; Sema verifies the canonical definition named by a reference in that message, while binding the message to its sender remains external.

Execution Authorization and Message Admission Two concurrent systems address agent safety at layers adjacent to Sema. Faramesh [56] places a non-bypassable *Action Authorization Boundary* between agent reasoning and real-world execution: agent intents are normalized into a Canonical Action Representation and every effectful action is gated by a deterministic PERMIT/DEFER/DENY decision, with fail-closed defaults and replayable decision records. CLBC [57] addresses covert signaling between LLM agents: messages only enter the shared transcript if a verifier approves them against a pinned predicate, yielding certified upper bounds on hidden coordination through otherwise policy-compliant text. Both are complementary to the Pattern Card handshake: Faramesh authorizes *what* an action may do at execution time, CLBC authorizes *which messages* may be spoken, and participating agents that resolve and verify the same full Pattern Card references can establish canonical-definition identity for the actions and capabilities they coordinate around. Together they form a three-layer safety stack: definition-identity verification (Pattern Cards), message verification (CLBC), and execution authorization (Faramesh).

The Synthesis: Code, Cognition, and Economy Most frameworks specialize in one domain: Knowledge Graphs handle static facts (Knowing), Agent Frameworks handle tool execution (Doing), and Crypto Protocols handle value (Transacting). Sema’s agent-coordination application sits at the intersection of these three disciplines. It treats **Thinking** (e.g., Reason#81ec) and **Doing** (e.g., Bid#79cf) as nodes in the same graph, allowing agents to reason about their economic commitments with the same grammar they use to reason about the world. The content-addressing principles are shared with IPFS [5] and Unison [6]. Unison deserves particular note: it identifies every function definition by the hash of its de-named abstract syntax tree, creating the kind of content-addressed definition DAG that the behavioral-contract realization applies to behavioral specifications. Unison’s separation of a human-readable name from a content-derived hash parallels Sema’s substrate/overlay separation: renaming an object’s own unhashed handle preserves its digest, while renaming handles retained inside hashed references can change dependent identities. Pattern Cards extend Unison’s insight from code definitions to behavioral definitions, adding typed interfaces and a fail-closed handshake.

Portable Capability Definitions The closest surface analogue to Sema’s Pattern Cards is Anthropic’s Agent Skills specification [58], released as an open standard in December 2025 and adopted across Claude, OpenAI, Microsoft, and Cursor. Agent Skills and Pattern Cards both place a structured header over a human-readable body, but three differences determine their verification properties. **Identity:** Agent Skills are identified by string name; two independently authored pdf-processing skills may differ, while Pattern Cards use content-derived identifiers, so divergence produces distinct hashes. **Contract:** Agent Skills bodies are freeform Markdown instructions; Pattern Cards carry structured, hashable contracts, making identity of a resolved canonical contract a digest comparison rather than a runtime judgment about whether two texts mean the same thing. **Composition:** Agent Skills are isolated artifacts; Pattern Cards reference other patterns by hash, so the Merkle root captures the transitive closure of the vocabulary a pattern relies on. The same comparison applies to adjacent industry formats: A2A Agent Cards [49] use human-assigned skill names, and MCP tool schemas [48] validate inputs and outputs but carry no hashable behavioral contract. These formats share the portable-capability goal with Pattern Cards; the latter add verifiable definition identity and explicit, hash-bound dependency composition.

Structured LLM Frameworks and Contract-Based Design Sema’s Pattern Cards—with preconditions, postconditions, invariants, and typed interfaces—exist within a broader lineage of structured LLM programming. DSPy [14] introduced typed signatures for language model pipelines, and DSPy Assertions [15] added computational constraints (`dspy.Assert`, `dspy.Suggest`) that are structurally similar to Pattern Card contracts. White et al. [16] provided the foundational work on formalizing prompt patterns using software design pattern structure. Leoveanu-Condrei [59] adapted Design by Contract to LLM agents, positing that agents satisfying the same contracts are functionally equivalent—the closest published work to Sema’s contract-based approach. Sema’s contribution relative to this lineage is not the contract structure itself but the cryptographic binding: hashing a contract into its identifier makes identity of the resolved canonical contract verifiable by digest comparison; it does not establish that differently worded contracts are semantically equivalent or that a runtime satisfies the contract.

Failure Analysis and Verification The MAST framework [31], analyzing 1,600+ execution traces across seven MAS frameworks, found that 79% of failures originate from specification and coordination issues—the strongest published empirical evidence for the problem Sema addresses. AgentSpec [60] defines formal semantics for runtime rule enforcement in LLM agents, operating at the execution layer where Sema operates at the semantic layer.

Classical Vocabulary Alignment The vocabulary alignment problem addressed by Sema has a multi-decade research lineage in multi-agent systems. Chocron and Schorlemmer [32] developed formal interaction-based alignment techniques for heterogeneous agent vocabularies. Sema addresses a narrower point in this lineage: once agents use a shared content-addressed definition, comparison of that external definition becomes deterministic hash-based identity verification. It does not align different but semantically equivalent vocabularies; reconciling those remains explicit alignment work.

Emerging Agent Infrastructure The W3C WebAgents Community Group [61] published an interoperability report synthesizing classical MAS research (FIPA, BDI) with modern LLM-based agent protocols, providing the most authoritative analysis of the standardization landscape Sema enters. Josifoski et al. [62] frame AI agents as modular “semantic processors” exchanging typed messages—conceptually close to Sema’s pattern-based meaning exchange. Microsoft’s NLWeb [63], led by the creator of Schema.org, uses Schema.org as a semantic foundation for agent-web interaction and exposes every instance as an MCP server, representing the most prominent effort to make web semantics machine-accessible for agents.

8.1 Contribution Summary

Against this landscape, Sema makes seven contributions, separable into the representation-agnostic communication primitive and its JSON/Merkle realization (C1), the Pattern Card agent-coordination layer (C2–C6), and bootstrap-library design (C7).

- C1: Hash-as-word inversion.** Sema moves content-addressed identity into text-carried references that can function as words, independent of the representation being hashed. This paper realizes the primitive with canonical JSON/Merkle hashing. In the Pattern Card layer, a reference such as `Delegate#00fa` combines a readable handle with a short locator that must resolve to, and be verified against, the full identifier for collision-resistant assurance.
- C2: Hashable behavioral contracts.** In the Pattern Card layer, the hashed object is a behavioral contract: mechanism, invariants, preconditions, postconditions, typed dependencies, and failure modes. Full-digest verification establishes identity of canonical form, not semantic equivalence; two behaviorally equivalent contracts with different wording still produce different hashes.
- C3: Field-level comparison.** Pattern Cards are hashed field-by-field and rolled up through a Merkle tree. Field hashes allow agents or tools to compare selected fields independently; coordinating on a matching subset requires a separate policy, while the current handshake remains fail-closed on root mismatch.
- C4: Identity separated from taxonomy.** Taxonomic metadata (`path`, `tier`, `related`) is excluded from the hashed surface. Patterns can be reclassified without changing their identity: the substrate is mathematics; the overlay is politics.
- C5: Fail-closed semantic handshake.** Discovery can be fuzzy, but collision-resistant coordination requires equality of full identifiers or full roots. In strict mode, `sema_handshake()` proceeds only on a full match and halts on mismatch; cooperative prefix mode provides drift detection rather than full assurance. The guarantee applies to compliant deployments that invoke and enforce the handshake.
- C6: Compositional vocabulary.** Patterns reference other patterns by hash, forming a wired DAG of behavioral specifications rather than a flat dictionary. Resolving dependencies exposes the cognitive stack a pattern relies on; execution still requires implementations or tools layered on top.
- C7: Bootstrap library and Grammar of Agency.** The paper provides an initial 457-pattern vocabulary organized by dependency layer and by agency role: actions, data shapes, and coordination/evaluation primitives. This is a second-order contribution: the communication primitive is representation-agnostic, but coordination still needs a shared Schelling point.

9 Limitations and Future Work

Several limitations constrain the current system and point toward future research.

Demonstration Scale. The multi-agent evaluation (Section 7.7) used $N = 5$ runs per condition with three agents. While the results are directionally consistent, larger-scale experiments with diverse agent architectures and task domains are needed to establish statistical significance and generalizability.

Layer Direction: Residual Judgment Calls. A handful of patterns sit at layer boundaries where the mechanism has genuinely mixed scope; we adjudicate these manually rather than algorithmically. The mechanism-sufficiency test (Section 4) narrows the judgment but does not eliminate it.

Specified but Not Built. Several Living Taxonomy mechanisms (Section 5) remain specifications rather than running code. The Judge#704c primitive has no scoring implementation, so quality assessment depends on manual review or external LLM evaluation. Pattern creation remains initiated through `sema_mint`; the Genesis Loop is not automated. The Experience Matrix proposes content-addressed tensor transfer but lacks serialization infrastructure (Section 5.4). These speculative mechanisms require experimental validation before deployment.

Historical Version Storage. A full Sema ID remains a content identity, but it can be resolved and verified only while a registry retains the corresponding content. The current GraphStore indexes one active definition per handle and does not archive replaced versions. Direct minting therefore rejects a parent replacement with inbound exact pins, while `sema_apply` and `pull` validate the complete projected corpus before permitting a reviewed batch. Keeping old pins resolvable requires a future multi-version content store. The unhashed `_meta.supersedes` field records replacement claims but is not itself an archive or a traversal implementation.

Built but Not Enforced. The fail-closed handshake is implemented for single-pattern and pattern-set verification through the MCP server, but is not enforced at the coordination-protocol level—agents can bypass it simply by not calling the handshake tool. As discussed in Section 3.4, fail-closed is a property of compliant deployments rather than an unconditional property of the protocol. Mandatory enforcement would require integration into the agent’s execution harness, not just its tool set—a deployment choice beyond the scope of Sema itself.

The Sema Tax. Condition B suggests that vocabulary lookup and invariant checking can introduce cognitive overhead. This potential “Sema Tax” remains relevant for latency-sensitive applications. Condition C had lower mean wall-clock time, but the demonstration does not establish that its prescribed protocol caused the difference. Future work should quantify this overhead across model families and context-window sizes.

Single-Agent Reasoning Scaffold. Although this paper presents Sema as inter-agent coordination infrastructure, the vocabulary’s Mind layer may also serve as an internal scaffold for individual agents: resolved reasoning patterns can carry explicit failure modes and invariants into the agent’s own context. This use case is not evaluated here; the theoretical foundations for composing reasoning across solvers are developed in [18].

Empirical Pattern Validation. The methodology described in Section 4 validates patterns analytically—by review, rules, guidelines, database/DAG constraints, and broad-use tests—but not empirically. The current evaluation does not measure whether a given pattern, once minted, actually improves downstream agent behavior: whether tasks coordinated through `StateLock#c9c2` succeed more reliably than those coordinated through ad-hoc prose, whether a Mind-layer reasoning pattern produces better solutions than unstructured chain-of-thought, or whether a given formulation of a mechanism is clearer to agents than its alternatives. An empirical layer would treat each pattern as a testable hypothesis, A/B-comparing variants on realistic agent benchmarks and feeding the results back into the next round of mechanism refinement. The refinement pass described in Section 4.8 sharpens patterns against shared formal criteria; an empirical refinement stage would sharpen them against observed task outcomes, producing a population of patterns that are not merely internally coherent but demonstrably useful.

Embedding Granularity. Structural distinctness analysis relies on `all-MiniLM-L6-v2` (384-dimensional vectors), a lightweight model optimized for speed over nuance. Domain-specific embedding models or larger sentence transformers may reveal finer-grained semantic clusters not visible at the current resolution. The 24 high-similarity pairs (≥ 0.70) warrant manual review to determine whether they represent genuine near-duplicates or appropriately related variants.

Definition Quality. When agents resolve the same full Pattern Card identifier and verify the returned content, they establish that they hold the same canonical definition, not that the definition is correct. A perfectly hashed but poorly specified pattern—one whose invariants are too weak, whose failure modes are incomplete, or whose mechanism is subtly wrong—propagates bad coordination just as reliably as good coordination. The cryptographic guarantee concerns canonical-content identity, not semantic correctness. The Judge#704c primitive and adversarial hardening

(Section 7.5) are designed to catch specification errors before minting, but both remain partially unimplemented. In practice, definition quality depends on the care of the author, and the protocol provides no defence against a well-hashed bad idea.

The Synonymy Limit. Full-reference verification establishes identity, not canonical uniqueness. Two Pattern Cards that describe behaviorally equivalent mechanisms in different words—different field orderings notwithstanding canonicalization, different invariant phrasings, different decompositions into primitives—will produce different hashes and therefore count as different patterns. It does not guarantee that holders of behaviorally equivalent meanings hold the same hash. Detecting and merging such synonyms is a curation task supported by the Taxonomist workflow and the embedding-similarity audits described in Section 4, not by the protocol itself. Refinement reduces synonymy in the bootstrap library, but content addressing remains agnostic to it: a community fork producing patterns equivalent to the canonical library would coexist with rather than collide with that library, and merging the two requires explicit alignment work outside the cryptographic layer.

Vocabulary Governance. The paper describes permissionless minting but does not address the governance challenges this creates at scale: namespace pollution, adversarial pattern injection, or quality degradation in the absence of curation. The Judge#704c primitive is designed to address this, but its effectiveness as an automated quality gate remains unvalidated.

Alignment Positioning. A content-addressed vocabulary of *beneficial* strategies—patterns for conflict de-escalation, informed consent, non-coercive persuasion, care under uncertainty, adversarial input handling—would carry the same guarantees the coordination library already provides: verified hash equality identifies the same canonical behavior specification, typed dependencies expose composition, and a compliant handshake can halt on mismatch. Building such a library is a straightforward application of the protocol presented here, and we expect it to be a fruitful direction.

That precision is neutral with respect to value alignment: a shared canonical definition need not specify benign behavior. The same vocabulary that lets aligned agents coordinate beneficial strategy definitions precisely would let misaligned agents coordinate harmful ones equally precisely. The safety of a Sema-equipped coordination stack is therefore bounded above by the alignment of the models running inside it: content-addressing supplies precision, and the underlying agents determine what that precision is used for. The metacognitive training proposal in [25]—weaving beneficial values into the model’s cognitive architecture through $P(\text{text}, \text{thinking}|\text{context})$ training—addresses that distinct alignment problem and is complementary to Sema. Underlying model alignment remains the foundation; a beneficial-strategy library supplies vetted patterns for runtime use.

10 Conclusion: The Substrate, Not the Law

Sema is a minimal communication primitive: hash information under an agreed representation and carry its content address in communication. This paper realizes the primitive through canonical JSON/Merkle hashing, instantiates it as Pattern Cards, and supplies a 457-card bootstrap library.

In the Pattern Card application, hashing a definition anchors a reusable word. Changes to hashed content reveal divergence, so an enforced handshake can catch mismatch before coordination. The result is compact reference backed by verification, precise definition coordination, and permissionless vocabulary growth.

Familiar systems such as Git, IPFS, Unison, and SDH use content addresses as infrastructure identifiers, citations, or formal references. Sema’s distinctive use is to make readable, hash-anchored terms part of the natural-language messages agents exchange: the unit of verification enters the unit of communication. The closest historical analogy is *movable type*, which made standardized, reusable, composable units native to the medium itself. Pattern Cards play an analogous role for machine-readable definitions—words backed by verifiable content addresses that agents can permissionlessly mint and recombine. Any channel that carries text can carry them, and their network effects can compound with vocabulary size.

The initial vocabulary is not a law but a seed: 457 patterns forming one concrete bootstrap ontology from which agents can coordinate and mint their own concepts.

Refining this library treated ontology maintenance as architectural engineering. Within its four-layer design, structural dependency proved more coherent than subject matter as a basis for classification: topology trumps topic, though whether that lesson generalizes remains empirical.

Sema holds no authority over what information may be addressed or how vocabularies are organized. Users can mint alternatives and let namespaces evolve. The project succeeds not if this vocabulary endures, but if the primitive enables a flourishing, self-governing commons of thought.

Same verified digest, same hashed content. Different verified digest, halt and clarify.

Code and Artifact Availability

The vocabulary and interactive graph are available at <https://semahash.org>. Code, paper source, vocabulary data, and release artifacts are available at <https://github.com/emergent-wisdom/sema>. The installable package is published at <https://pypi.org/project/semahash/>, and its MCP Registry entry is at <https://registry.modelcontextprotocol.io/servers/io.github.emergent-wisdom/semahash>. The version series is archived at doi:10.5281/zenodo.19548971.

A Pattern Card Specifications

This appendix contains the automatically generated specifications for the core vocabulary.

The following cards show the **canonical hashed content** for each pattern. Only semantic fields are included—these are the exact fields that produce the Merkle root hash. Metadata fields (handle, _meta, tier, etc.) are *not* part of the hash.

A.1 sema:StateLock#mh:SHA-256:c9c2809bdf97bb6ac17a8ffe428d4aa9a28294b99d64cc8a336a239d26a8afb8

```

1 {
2   "dependencies": {
3     "composes_with": {
4       "backoff": "sema:Backoff#mh:SHA-256:9
5         e59718422f5408d7892feccb6d538a21103f9573b122b891ee3c8cf66303e72",
6       "cooldown": "sema:Cooldown#mh:SHA-256:
7         c288d88055cb0aae80c4b67ed62afcd3ebcc5de18749fc6efb15e9f8c529b9df"
8     },
9     "references": {
10      "actor": "sema:Actor#mh:SHA-256:86
11        fc0930b97ffb4fe85d4820af8e17a43166b745d181895f576edf39edde6aa8",
12      "lock": "sema:Lock#mh:SHA-256:95
13        c2ee952a5301d4b346a5e8693350829d88b3c600048eedcf87bb00c23ee5fb",
14      "state": "sema:State#mh:SHA-256:4
15        f25e6a16ccd37dc0bc154cccc7055cb504d49daa8b712f372a425eb6c7b8cb2"
16    }
17  },
18  "signature": [
19    "Lock(State)"
20  ],
21  "mechanism": "A coordination pattern where two {{actor}}s temporarily 'fuse' a
22    subset of their writable {{state}}. During the {{lock}}, changes require a
23    cryptographic signature from both. Contention triggers {{backoff}} and {{
24    cooldown}}.",
25  "gloss": "Atomic coordination via temporary state fusion",
26  "invariants": [
27    "Both {{actor}}s sign every write to the fused {{state}} subset.",
28    "Exclusive over that subset: no third party writes it while the {{lock}} holds
29    .",
30    "Temporary: the fusion ends on completion or timeout, never implicitly."
31  ],
32  "failure_modes": [
33    "Deadlock: both parties wait for each other indefinitely.",
34    "Livelock: lock acquired and released rapidly, denying access to others.",
35    "Premature dissolution: state auto-dissolves while legitimate work is in
36    progress."
37  ]
38 }
```

28 }

A.2 sema:FractalIntelligence#mh:SHA-256:22a03a3c350a73162e27b441004659f4540eaf1a7cb3a6ca01fce15c43af869c

```

1 {
2   "dependencies": {
3     "composes_with": {
4       "conceptual_decomposition": "sema:ConceptualDecomposition#mh:SHA-256:40
        d9cc8428747ad32e921a0317e560d14031e656cfd10089992ddf161dc0e450",
5       "localized_learning": "sema:LocalizedLearning#mh:SHA-256:868
        bb747edec043a9ff7b7716e716d4269b635af0945c73dc970c53aa7595b6b",
6       "marginal_value_rule": "sema:MarginalValueRule#mh:SHA-256:
        ffe1bb67e5d273f3acf3cb92876189b68752befd2ab6d0bfa406fb1929c4a17b",
7       "polymorphic_solver": "sema:PolymorphicSolver#mh:SHA-256:28
        f1d5d9b3edcc3967139a056b71acd75050f65e7002644e5a837dd5d9507a07",
8       "problem_framer": "sema:ProblemFramer#mh:SHA-256:27
        c36c446e3590bab1651a989dbcb9013966ecea554ced75bfd9cf1642a8996",
9       "reason": "sema:Reason#mh:SHA-256:81
        ec8196c7c1c761fe353c8d26463bcb61b54f844aa3422a40019f82ef202321",
10      "recursion_dive": "sema:RecursionDive#mh:SHA-256:54763
        d3b1a9745e7f4a13d34b68355866c9cad797d786c8c43cb0306a37e78d3",
11      "reframe": "sema:Reframe#mh:SHA-256:
        cbfd4fadde729367ef8f32a68951e5852535b7f7c3edc68a502d9187b8c1d54a",
12      "state_snapshot": "sema:StateSnapshot#mh:SHA-256:0
        a0fee933e452552f3128e0f5491a74cdb132c8417dfc3cf2e6f1e2141e0b21b",
13      "synthesis": "sema:Synthesis#mh:SHA-256:4
        f1fecba4183e6a79bfb3bd8f1981a6b6fb9f3dced92a484a9aa466c93ae249b"
14    },
15    "references": {
16      "agent": "sema:Agent#mh:SHA-256:
        e127459069739fffc96a71d54ef6e8c97ca53471d5349c16aa72a092a33756cd",
17      "conservation": "sema:Conservation#mh:SHA-256:0
        b32d007b27a9b5201c46931bdbabd09d08ff9d7b01cf49469f61709989507c2",
18      "experience_sharding": "sema:ExperienceSharding#mh:SHA-256:
        c8a197339886a6bbec3201ead589ad7fd37bbfcde46d6470df59ded8aaa80153",
19      "specialize": "sema:Specialize#mh:SHA-256:916
        a25154459e500447146456a53cf4bc39641c120f8323554b53561be571d77",
20      "strategy": "sema:Strategy#mh:SHA-256:3
        dc598d0c6af12389df445c204e17df03950dbc886656b85700e6c87c31b2eef",
21      "system": "sema:System#mh:SHA-256:
        f8eb318a1113a10fead02fbf14ae433767ac1a69c9579cdb584cc622068e90b3",
22      "task": "sema:Task#mh:SHA-256:
        a85010f508f85055170c396c8fa390125c1206888c565ac10651f3969a339f8d",
23      "universal_solver_tree": "sema:UniversalSolverTree#mh:SHA-256:32
        f92696f93c53120145b8dc786a24571f0114d90ffe472f6484998451553258"
24    }
25  },
26  "signature": [
27    "System(Reason)"
28  ],
29  "mechanism": "Expansion of cognitive capability through {{
    conceptual_decomposition}}: a concept (problem or task) is broken into
    contract-bound sub-concepts, each governed by the same five-surface Solver
    Contract (Manifest, Execute, Consult, Verify, Feedback) that governs the
    parent. A few {{agent}}s can assign solver roles to themselves and perform
    lightweight fractal intelligence for a specific problem \u2014 the resulting
    structure may persist as a reusable pattern that improves through use, or may
    be torn down at completion; both are legitimate modes. The unified {{system
    }} of scalable cognition uses {{reason}} to orchestrate fractal expansion
    within the {{universal_solver_tree}}. A {{problem_framer}} initiates by
    formulating a high-level {{strategy}} before assigning a {{polymorphic_solver
    }} to a {{task}}; the solver executes a {{recursion_dive}} to spawn child

```

```

    nodes, each applying {{specialize}} with {{localized_learning}}, while {{
    experience_sharding}} and {{synthesis}} preserve global coherence. {{
    state_snapshot}} provides crash recovery for persistent instances. {{
    marginal_value_rule}} governs recursion depth. On failure, {{reframe}}
    restructures the tree.",
30 "gloss": "Expansion of cognitive capability through recursive decomposition of
    concepts into contract-bounded sub-concepts",
31 "invariants": [
32   "Fractal Self-Similarity: The process at the Root is identical to the process
    at the Leaf.",
33   "Bounded Expansion: Recursion is limited by Economic constraints (Marginal
    Value).",
34   "Memory {{conservation}}: Specialization must not result in the loss of global
    context."
35 ]
36 }
```

A.3 sema:SpectralTune#mh:SHA-256:c0aab008d2cd2f653e56508356987477f9207b4cf6435f4416105b3f37bd544a

```

1 {
2   "dependencies": {
3     "accepts": {
4       "signal": "sema:Signal#mh:SHA-256:6
        eaff8a06e08e75279670bc09d88175d130815240fdb3a21252c0364769c1fc"
5     }
6   },
7   "mechanism": "Instead of sending a message and hoping it is understood, the
    sender transmits a 'tuning {{signal}}'\u2014a sequence of hash-based
    challenges representing the semantic context (ontology, assumptions, version)
    . The receiver must 'resonate' by proving they hold the matching semantic
    context.",
8   "gloss": "Verifying ontology alignment before data transfer",
9   "invariants": [
10    "No payload data is processed before Tune_ACK.",
11    "On hash mismatch, halt and escalate rather than retry."
12  ],
13   "preconditions": [
14    "Both agents possess valid hash function H"
15  ],
16   "postconditions": [
17    "Channel is established with matching context hashes, OR connection terminated
    and escalated"
18  ],
19   "parameters": [
20     {
21       "name": "context_chunks",
22       "type": "PositiveInteger",
23       "range": "unspecified",
24       "description": "Prompts to hash"
25     },
26     {
27       "name": "hash_algo",
28       "type": "String",
29       "range": "{BLAKE3, SHA256}",
30       "description": "Hash algorithm for semantic context verification"
31     }
32  ],
33   "failure_modes": [
34    "Over-strict halt: a benign version skew or encoding difference produces a
    hash mismatch and halts a channel whose parties would have understood each
    other. Hash equality admits no partial match, so any difference is total."
35  ]
36 }
```

36 }

A.4 sema:SteelmanCheck#mh:SHA-256:bd611ca0ddfecaf9d70fd83ff2b51a6a3b121e95936c586cc5a4dfd3fa7167bf

Note: This pattern was downgraded to **Tier 2** following adversarial analysis.

```

1 {
2   "dependencies": {
3     "composes_with": {
4       "check": "sema:Check#mh:SHA-256:6890
          b221c38531710528cd84bbf3a957e29d9836dfc9c02669fc0a86709bcb00",
5       "critique": "sema:Critique#mh:SHA-256:646
          b71acb10a6b7f0874f3d5c7977c44d253f29f9c1231c974d85c7fba131980",
6       "judge": "sema:Judge#mh:SHA-256:704
          cbb71588d7a3bafa08b71c863005e31ec4400860b9cb8b094e660da9c7c6c"
7     },
8     "references": {
9       "agent": "sema:Agent#mh:SHA-256:
          e127459069739fffc96a71d54ef6e8c97ca53471d5349c16aa72a092a33756cd",
10      "belief": "sema:Belief#mh:SHA-256:
          e160f5d3c82f6a64902c8619ec411f1c33eb0a4d031aea46bddb0f0776d5f150",
11      "cognitive_bias": "sema:CognitiveBias#mh:SHA-256:
          c8a4abe2e0ce048be16d25f890a50f8c7095e099b5ab2de23aa059942328f387",
12      "decision": "sema:Decision#mh:SHA-256:
          b1407127acd4a39314fc24b418310e4b95e0347e4e22f376151ab765e2dfd2c7",
13      "loop": "sema:Loop#mh:SHA-256:37
          f953c0b21b9b0c72a639c4e1a54354faa71f07905531d271f6bbf40c341603",
14      "robustness": "sema:Robustness#mh:SHA-256:5304
          c7824b76c6f308bb1a33ba6171d41fd061a5b104f21710bbb6c7a7c85b43"
15    },
16   },
17   "signature": [
18     "Check(Robustness)",
19     "Critique(Belief)",
20     "Judge(Critique)"
21   ],
22   "mechanism": "Before finalizing a {{decision}} or output, the {{agent}} MUST
          generate a claim-specific counter-argument to its conclusion. It uses {{
          critique}} to challenge the underlying {{belief}}, {{judge}} to score the
          counter-argument, and {{check}} to test the {{decision}}'s {{robustness}}
          against any counter-argument that meets 'strength_threshold'. A sub-threshold
          counter-argument is strengthened up to 'iteration_limit'; if no adequate
          counter-argument is produced, release is blocked. Once an adequate counter-
          argument exists, a Falsified robustness result requires revision or discard,
          Unknown blocks release pending review, and only Verified permits finalization
          . It prevents confirmation {{cognitive_bias}}. For adversarial contexts, see
          adversarial steelmanning.",
23   "gloss": "Qualify the opposition, then release only if the conclusion remains
          robust",
24   "invariants": [
25     "Adequate opposition: before release, {{judge}} scores the counter-argument at
          or above 'strength_threshold'.",
26     "The counter-argument identifies the specific claim it contradicts.",
27     "Release discipline: only a Verified {{check}} of the {{decision}}'s {{
          robustness}} after adequate opposition permits release; Falsified requires
          revision or discard, and Unknown requires review."
28   ],
29   "preconditions": [
30     "A {{decision}} or output exists to challenge.",
31     "The conclusion exposes a specific claim or rationale that can be contradicted
          ."
32   ],
33   "postconditions": [

```

```

34     "Either bounded failure to produce an adequate counter-argument blocks release
      , or an adequate counter-argument and terminal {{check}} result are
      recorded; the {{decision}} is finalized only on Verified."
35 ],
36 "parameters": [
37   {
38     "name": "iteration_limit",
39     "type": "Integer",
40     "range": "[1, 5]",
41     "description": "Maximum strengthening attempts before failure to produce
      adequate opposition blocks release"
42   },
43   {
44     "name": "strength_threshold",
45     "type": "Float",
46     "range": "[0.7, 0.95]",
47     "description": "Minimum {{judge}} score for a counter-argument to qualify as
      a steelman; it is not the verdict on the {{decision}}"
48   }
49 ],
50 "failure_modes": [
51   "Paralysis by analysis (stuck in {{critique}} {{loop}}).",
52   "Collusion: Proposer and Critic are same entity; susceptible to Strawman Waltz
      attack where agent generates weak counter-arguments to easily defeat them
      ."
53 ]
54 }

```

A.5 sema:OptimisticSolver#mh:SHA-256:0dcabc92d9b6bbac524d5e7445ecd0590872956e3e392304e6de24655dbea4f7

Note: This pattern was downgraded to **Tier 2** following adversarial analysis.

```

1 {
2   "dependencies": {
3     "composes_with": {
4       "atomic_bid": "sema:AtomicBid#mh:SHA-256:005
      abad4abe486ee0be82fdf09f7e4787c05ae1bc7326442bf8834cca4856cd3",
5       "compensate": "sema:Compensate#mh:SHA-256:4
      b87623f38564fcc95b87d2cbf5df2f5ff41ef27e617ccafd76c7b7896b144e7",
6       "compute_budget": "sema:ComputeBudget#mh:SHA-256:4091
      ac7c6677342920dc53808d02830e35c5849f2af1bb9ba7749c576d37478e",
7       "pathway_memory": "sema:PathwayMemory#mh:SHA-256:
      b15f1dd794db8b776de7646d2431466934bb901c3d443e0c92e85b58ad3c3ca5",
8       "reflexion": "sema:Reflexion#mh:SHA-256:81
      eb9140703ea95f53556741df731af84c1cada66cc1e0f3a1d88e4de5fe54d8"
9     },
10    "references": {
11      "parallel": "sema:Parallel#mh:SHA-256:
      d58d6696be11be09651e71307f0ea6f42bb45fac137ed985a61e475144f687a5",
12      "polymorphic_solver": "sema:PolymorphicSolver#mh:SHA-256:28
      f1d5d9b3edcc3967139a056b71acd75050f65e7002644e5a837dd5d9507a07",
13      "rigorous_solver": "sema:RigorousSolver#mh:SHA-256:93
      b824003c5454397626a8c2afe22a03c0b5dae890b8a7bc2158f8139825bd96"
14    }
15  },
16  "mechanism": "A high-velocity implementation of {{polymorphic_solver}} designed
      for efficient multi-agent coordination. Requires a {{parallel}} runtime (
      Actor Model with Mailboxes) to prevent serial deadlock. It explicitly couples
      the standard Solver lifecycle (Reason -> Solution) with the {{atomic_bid}}
      protocol. Unlike the base abstraction, this pattern MANDATES that the agent
      plan and execute in a single turn. It relies on {{reflexion}} and {{
      compensate}} for error correction rather than pre-action permission. Use {{
      compute_budget}} to bound resource consumption. Contrast with {{

```

```

    rigorous_solver}} which prioritizes safety over speed. A {{pathway_memory}}
    accumulates across runs so the optimistic route-selection converges on
    strategies that historically succeeded under similar conditions.",
17 "gloss": "High-velocity solver requiring parallel runtime",
18 "invariants": [
19   "Turn Atomicity: Must output [BID] and [TOOL] in the same response.",
20   "Non-Blocking: Cannot wait for Orchestrator approval on standard tasks."
21 ],
22 "preconditions": [
23   "Runtime supports Asynchronous/Parallel message delivery (Actor Model).",
24   "Message Bus is non-blocking (Mailbox pattern)."
25 ],
26 "failure_modes": [
27   "Over-Eager Execution: Solving the wrong problem efficiently because feedback
    was skipped.",
28   "Serial Deadlock: If deployed on a single-threaded (Talking Stick) engine,
    multiple atomic outputs will be dropped, halting the swarm."
29 ],
30 "extends": "sema:PolymorphicSolver#mh:SHA-256:28
    f1d5d9b3edcc3967139a056b71acd75050f65e7002644e5a837dd5d9507a07"
31 }

```

B Category Taxonomy

This appendix presents the complete taxonomic hierarchy of the bootstrap library.

The 457 patterns are organized into 12 category names across 13 layer-category paths and 4 fundamental layers:

Physics Layer (16 patterns) The immutable substrate.

- **Primitives** (15): Attractor, Causation, Conservation, Dampen, Decay, Distance, Entropy, Equilibrium, Gradient, Lock, Measurement, MutualInformation, Noise, PhaseTransition, Reversibility.
- **Time** (1): CausalBarrier.

Infrastructure Layer (158 patterns) Operational constraints.

- **Data Structures** (96): AcceptSpec, Aesthetics, Anomaly, Artifact, Assessment, Assumption, Audit, Ballot, Belief, Boolean, Break, Cache, Card, Category, Chain, CognitiveBias, ConceptAnchor, Condition, Constraint, Context, Contract, Correlation, Criteria, Cyclic, DAG, Datum, Decision, Event, Exception, ExecutionManifest, FailureTrace, Forest, FrameSpec, Goal, Hierarchy, Hypothesis, Identity, Ledger, MECE, MechanisticDesign-Proposal, Message, Meta, Metric, MetricReading, Mode, Nature, Option, Outcome, Overlap, Parallel, PerformanceSignal, Permission, Plan, Probability, Problem, ProblemSpace, Prompt, Proposal, ProtoPack, Protocol, Prototype, Queue, Resource, Result, Risk, Role, RolloutManifest, RuleSet, Score, ScoringFunction, Sequence, Shard, Signal, Skeleton, Snapshot, Solution, SolverManifest, Spec, State, Status, Step, Stream, StyleSpec, Subject, Summary, System, Task, Tension, Topology, Trait, Transition, Tree, Value, Variable, Vector, Work.
- **Primitives** (53): Act, Actor, Aggregate, Backoff, Branch, Budget, Care, Check, CircuitBreaker, Combine, Compare, Compensate, Compress, Cooldown, EntropyPump, Equivalence, ExponentialBackoff, FailClosed, Feedback, FeedbackSignal, Gate, Greet, Heartbeat, Hysteresis, IdempotentWrite, Incongruity, Judge, Loop, Monitor, MonitorReport, Mutex, NegativeProof, Observe, ParetoFront, Probe, Quorum, Rank, ReAttempt, Route, Sandbox, Search, Select, Sign, StateAudit, StateSnapshot, StateTransition, TaskLifecycle, Throttle, TimeWarpLog, ToolInvoke, Trace, TriGate, Warmup.
- **Verification** (9): AuditTrail, CompatibilityCheck, ExplainBeacon, HumanApprove, InputGuard, OathBind, OutputGuard, SpotAudit, Validate.

Mind Layer (181 patterns) Hybrid cognition—always decomposable and delegatable.

- **Inference** (22): BaseRateInclude, BayesUpdate, BreadthGovernor, ConfidenceCalibrate, ConfirmationBlock, ContextFirst, EpistemicCalibrate, HackDetect, HindsightBlock, LayeredCheck, NormCheck, NormativeJudge, OntologyAdapt, ProphetFanOut, RegimeSense, ScopeFreeze, SemanticTabu, SourceEvaluate, SurprisalUpdate, SurvivorCorrect, TemporalEnsembleForecasting, TruthseekingProtocol.
- **Memory** (15): BeliefTracking, ChunkMerge, ContextCompress, CurriculumReplay, ExperienceSharding, HolographicShard, LatentAttachment, LocalizedLearning, PathwayMemory, Proprioception, RetrievalAugment, Scratchpad, SelfReminder, SimulationTrace, TraceBelief.
- **Reasoning** (63): Abduction, Axiom, BackwardChain, Bisect, ChainOfThought, CiteBack, CognitiveEcho, CollaborativeWritingProtocol, ConceptualDecomposition, ConstructOntology, Critique, Decompose, DecompositionGate, Deduction, DeepResearch, Dialectic, Eliminate, EpistemicCascade, Estimate, EthicalReasoningProtocol, Expansive, ExtendedThinking, Fermi, FirstPrinciples, FrameError, Generalize, GraphOfThought, HeuristicSnap, HumanEmulatorProtocol, Induction, Interpret, Invert, LeastToMost, LivedProof, MetaPrompt, Parsimony, PatternDiscovery, PatternEmergence, ReAct, Realizable, Reason, RecursionDive, RecursiveRootCause, Refine, Reflexion, Reframe, RequestFraming, SelfConsistency, SkeletonOfThought, SocraticLoop, Specialize, SteelmanCheck, StepBack, StrategicReading, Summarize, Synthesis, Think, Translate, TreeOfThoughts, Uncertain, Understand, Verification, WhyClimb.
- **Strategy** (81): AdversarialSteel, Agent, AnalogyBridge, AntifragileInversion, BeamSearch, Bubble, Build, CapacityPressure, CommitmentDevice, Compose, ComputeBudget, ConceptBlend, ConstraintFirst, ContingencyPlan, Creative, CreativeBlend, Crystallize, Deep, Defer, DepthGovernor, DesignArchitect, DiscoveryProtocol, DogfoodFirst, EmpathySim, EmpiricalTest, EpistemicROI, EventReact, Experiment, ExploreExploit, Falsification, FractalIntelligence, HypothesisEngine, HypothesisLadder, Jester, Kairos, LatentWander, LateralOptimization, ManifestPlanning, MarginalValueRule, MentalSim, MetaCheck, MetaProtocols, MintWhenFriction, NoiseInjection, Novelty, OODA, OpportunityCost, OptimalStop, Optimize, PURE, PUREBrainstorming, PURECheck, PUREOptimization, Parallelize, PerspectiveEnsemble, PolymorphicSolver, PreMortem, Prioritize, ProblemFramer, RedTeam, Reflex, RegretMinimization, RepresentationSwap, Retry, RigorousSolver, Roadmap, RootSolver, SacrificialProbe, Satisfice, Silence, Simulation, Solver, SteelmanFirst, Strategy, SunkCostIgnore, TensionHold, ThinSlice, TimeboxThink, TradeOff, UncertaintyMap, WorldReversible.

Society Layer (102 patterns) Multi-agent coordination.

- **Coordination** (12): Compromise, Consensus, ConsensusFinder, Delegate, Disband, Elect, IdentityHandshake, LazyConsensus, OntologyHandshake, Rally, Resonate, Vote.
- **Economics** (9): AtomicBid, AttentionMarkets, Award, Bid, ContinuousResourceAuction, ExchangeRate, Gardener, ValuePeg, Yield.
- **Governance** (7): AnchorDrop, Constitution, DocumentedOverride, Responsibility, SolverTree, UniversalSolverTree, WorldTransparent.
- **Protocols** (74): AdversarialProof, AgentDiscover, AgentProtocol, AgentSandbox, AmbiguityResolution, BearerToken, BoundedTask, Canary, ConfusedDeputy, ContextSwitch, CounterfactualAnchor, DataMinimization, DeliberativeAlign, Deploy, Discover, DissentSeek, DriftWatch, EbbFlowSync, EjectionSeat, EvaluatorOptimizer, ExpiringToken, FabricSharding, FeatureFlag, GenealogicalTrace, GlacialVault, Global, GracefulDegradation, Handoff, HeldRelease, IntentGap, InternalConsistency, InvariantFilter, LatticeCommit, MemeticSeed, ModestClaim, MonotonicCounter, Nucleate, OptimisticSolver, Oracle, OrchestrationLoop, OsmoticFilter, PatternSketch, PermissionEscalate, PhasedRefinement, PromiseGraph, PromptChain, PropheticQuorum, QuorumPulse, RealizationProtocol, ReceptivityGate, ReversibilityCheck, Robustness, Rollout, RolloutWatch, RootHashGossip, ShoutWhisper, SignalReflection, SolverNode, SomaticMarker, SpectralTune, StateLock, Stigmergy, StructuralCoaching, SynergisticMode, Taper, ThreeLevelCollision, TieredAccess, ToolDiscovery, TranslationProxy, UniqueHandle, UptakeAsGround, UptakeOverTimestamp, WorkerMode, Workflow.

References

- [1] Elizabeth Gibney. Can researchers stop ai making up citations? *Nature*, 645(8081):569–570, 2025.
- [2] Mingqian Feng, Xiaodong Liu, Weiwei Yang, Jialin Song, Xuekai Zhu, Chenliang Xu, and Jianfeng Gao. SEMA: Simple yet effective learning for multi-turn jailbreak attacks. In *International Conference on Learning Representations (ICLR)*, 2026.
- [3] Sandro Hawke. Identification via secure definition hash. W3C Draft Proposal, 2002.
- [4] Tobias Kuhn and Michel Dumontier. Trusty URIs: Verifiable, immutable, and permanent digital artifacts for linked data. In *The Semantic Web: Trends and Challenges (ESWC 2014)*, pages 395–410. Springer, 2014. Graph-level content hashes embedded in dereferenceable identifiers for nanopublications.
- [5] Juan Benet. Ipfs - content addressed, versioned, p2p file system. *arXiv preprint arXiv:1407.3561*, 2014.
- [6] Paul Chiusano, Rúnar Bjarnason, et al. The unison language reference, 2024. Content-addressed code architecture.
- [7] Anders Rundgren, Bret Jordan, and Samuel Erdtman. RFC 8785: JSON canonicalization scheme (JCS), 2020.
- [8] Dave Longley, Gregg Kellogg, and Dan Yamamoto. Rdf dataset canonicalization: A standard rdf dataset canonicalization algorithm. W3c recommendation, W3C, 2024.
- [9] Joel Gustafson. Content-addressing semantic data. Knowledge Futures Group Notes, 2019.
- [10] Ben Laurie, Eran Messeri, and Rob Stradling. RFC 9162: Certificate transparency version 2.0, 2021.
- [11] Kunal Sawarkar, Abhilasha Mangal, and Shivam Raj Solanki. Blended rag: Improving rag (retriever-augmented generation) accuracy with semantic search and hybrid query-based retrievers. In *2024 IEEE 7th international conference on multimedia information processing and retrieval (MIPR)*, pages 155–161. IEEE, 2024.
- [12] Manoj Ghuhan Arivazhagan, Lan Liu, Peng Qi, Xinchu Chen, William Yang Wang, and Zhiheng Huang. Hybrid hierarchical retrieval for open-domain question answering. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 10680–10689, 2023.
- [13] Erica Coppolillo and Simone Mungari. Unexpected knowledge: Auditing wikipedia and grokipedia search recommendations, 2025.
- [14] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T Joshi, Hanna Mober, et al. DSPy: Compiling declarative language model calls into state-of-the-art pipelines. In *International Conference on Learning Representations (ICLR)*, 2024.
- [15] Arnav Singhvi, Omar Khattab, et al. DSPy assertions: Computational constraints for self-refining language model pipelines. In *Workshop on Systems for Next-Gen AI Paradigms (NeurIPS)*, 2024.
- [16] Jules White, Quchen Fu, Sam Hays, Michael Sandborn, Carlos Olea, Henry Gilbert, Ashraf Elnashar, Jesse Spencer-Smith, and Douglas C. Schmidt. A prompt pattern catalog to enhance prompt engineering with ChatGPT. In *arXiv preprint arXiv:2302.11382*, 2023. Foundational work on formalizing prompt patterns using software design pattern structure.
- [17] Jon Postel. RFC 761: DoD standard transmission control protocol, 1980. Originating statement of the robustness principle: “be conservative in what you do, be liberal in what you accept from others.” The shorter “be conservative in what you send, be liberal in what you accept” paraphrase is from RFC 1122 (1989).
- [18] Henrik Westerberg. Fractal intelligence: Conceptual decomposition as problem-solving infrastructure. Zenodo, 2026. Preprint. Introduces the Fractal Intelligence architecture: recursive conceptual decomposition governed by a marginal-value rule, with a five-surface Solver Contract.
- [19] Henrik Westerberg. The ontology of the alien: Escaping the median trap in LLM ideation. Zenodo, 2026. Demonstrated emergent metacognition in multi-agent ontology construction.
- [20] Stewart Brand. *The Clock of the Long Now: Time and Responsibility*. Basic Books, 1999. Source of the ‘Pace Layering’ concept applied to civilization constraints.

- [21] Holger Knublauch and Dimitris Kontokostas. Shapes constraint language (shacl). W3c recommendation, W3C, 2017.
- [22] Charles Babbage. *On the Economy of Machinery and Manufactures*. Charles Knight, London, 1832. Source of the Babbage Principle regarding the division of mental labor.
- [23] Elliot Meyerson et al. Solving a million-step llm task with zero errors, 2025. Empirical validation of the Babbage Principle via ‘Massively Decomposed Agentic Processes’.
- [24] Google DeepMind. AlphaEvolve: A gemini-powered coding agent for designing advanced algorithms, 2025. Discovered the 48-step matrix multiplication algorithm (beating Strassen’s 1969 record) and optimized global compute infrastructure.
- [25] Henrik Westerberg. The superintelligence that cares about us. Zenodo, 2025. July 2025. Proposed the ‘Living Taxonomy of Thought’ as a prerequisite for beneficial AI.
- [26] Zhengwei Tao, Ting-En Lin, Xiancai Chen, Hangyu Li, Yuchuan Wu, Yongbin Li, Zhi Jin, Fei Huang, Dacheng Tao, and Jingren Zhou. A survey on self-evolution of large language models, 2024.
- [27] Ali Behrouz, Meisam Razaviyayn, Peilin Zhong, and Vahab Mirrokni. Nested learning: The illusion of deep learning architectures. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. Foundational theory for separating ‘Fast Context’ from ‘Slow Structure’.
- [28] David Wadden et al. Fact or fiction: Verifying scientific claims. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7534–7550, 2020.
- [29] Dasha Metropolitansky and Jonathan Larson. Towards effective extraction and evaluation of factual claims. ACL 2025, 2025. Introduces Claimify, an LLM-based claim extraction method.
- [30] Frédéric Berdoz, Leonardo Rugli, and Roger Wattenhofer. Can AI agents agree?, 2026. Byzantine consensus study: LLM agents fail to reliably reach valid agreement even in benign no-stake settings; performance degrades with group size and collapses under a single Byzantine agent.
- [31] Mert Cemri et al. Why do multi-agent LLM systems fail? In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks*, 2025. Introduces the MAST taxonomy. Analyzed 1,600+ execution traces across 7 MAS frameworks; found 79% of failures originate from specification and coordination issues.
- [32] Paula Chocron and Marco Schorlemmer. Vocabulary alignment in openly specified interactions. *Journal of Artificial Intelligence Research*, 68:69–107, 2020. Formal interaction-based vocabulary alignment for multi-agent coordination.
- [33] Gottfried Wilhelm Leibniz. *Dissertatio de arte combinatoria*. Fickius, Leipzig, 1666. The foundational proposal for the ‘characteristica universalis’ and the derivation of logic from a namespace of prime concepts.
- [34] Craig Sayers and Kave Eshghi. The case for generating URIs by hashing RDF content. Technical Report HPL-2002-216, HP Laboratories, 2002.
- [35] Samuele Marro, Emanuele La Malfa, Jesse Wright, Guohao Li, Nigel Shadbolt, Michael Wooldridge, and Philip Torr. A scalable communication protocol for networks of large language models. *arXiv preprint arXiv:2410.11905*, 2024.
- [36] Christine Lemmer-Webber. Content addressed descriptors and interfaces with Spritely Goblins. <https://dustycloud.org/tmp/interfaces.html>, 2021. Draft specification for OCapN content-addressed interface discovery.
- [37] Solidity Contributors. Contract ABI specification. <https://docs.soliditylang.org/en/latest/abi-spec.html>, 2015. Defines 4-byte Keccak-256 function selectors as call routing tokens.
- [38] Ian Grigg. The Ricardian contract. https://iang.org/papers/ricardian_contract.html, 2004. First presented 1996; formalized hash-identified human-readable contracts.

- [39] Gabriella Gonzalez. Semantic integrity checks are the next generation of semantic versioning. <https://haskellforall.com/2017/11/semantic-integrity-checks-are-next>, 2017. Describes Dhall’s content-addressed normal-form hashing.
- [40] Patrick Lewis et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.
- [41] Yunfan Gao et al. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023.
- [42] Manu Sporny, Grant Noble, Dave Longley, Daniel C. Burnett, Brent Zundel, and Kyle Den Hartog. Verifiable credentials data model v1.1. W3c recommendation, W3C, 2022.
- [43] Pierre-Yves Vandenbussche, Ghislain A. Ateamezing, María Poveda-Villalón, and Bernard Vatant. Linked open vocabularies (lov): A gateway to reusable semantic vocabularies on the web. *Semantic Web*, 8(3):437–452, 2017.
- [44] Christopher Alexander, Sara Ishikawa, Murray Silverstein, Max Jacobson, Ingrid Fiksdahl-King, and Shlomo Angel. *A Pattern Language: Towns, Buildings, Construction*. Oxford University Press, 1977.
- [45] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [46] Peter Steinberger. OpenClaw: Personal AI assistant. <https://openclaw.ai>, 2026. Open-source autonomous agent framework with multi-agent routing and persistent execution.
- [47] Abul Ehtesham, Aditi Singh, Gaurav Kumar Gupta, and Saket Kumar. A survey of agent interoperability protocols: Model context protocol (MCP), agent communication protocol (ACP), agent-to-agent protocol (A2A), and agent network protocol (ANP), 2025.
- [48] Anthropic. Model context protocol. Anthropic, 2024. Open protocol for LLM-tool integration, donated to Linux Foundation December 2025.
- [49] Google. Announcing the agent2agent protocol (A2A). Google Developers Blog, 2025. Open protocol for agent-to-agent collaboration, donated to Linux Foundation June 2025.
- [50] Agent Network Protocol Community. Agent network protocol (ANP), 2025. Decentralized agent discovery and collaboration via DIDs and JSON-LD.
- [51] Otavio Serra. Symplex: Agent semantic protocol. <https://github.com/olserra/symplex>, 2026. MCP extension using semantic intent vectors (float32 embeddings) for agent interoperability via cosine similarity.
- [52] Ecma International. NLIP: Natural language interaction protocol. <https://ecma-international.org>, 2025. Ecma-430 through Ecma-434. Standardized protocol where agents use generative AI to translate between local ontologies, requiring no shared vocabulary.
- [53] Charles Fleming, Vijoy Pandey, Luca Muscariello, and Ramana Kompella. A layered protocol architecture for the internet of agents, 2025.
- [54] Skan AI. Agentic ontology of work (AOW): A common language for the age of intelligent automation. Skan.ai, 2026. Enterprise semantic framework defining 9 canonical entities for agent governance.
- [55] Zhenhua Zou, Zhuotao Liu, Lepeng Zhao, and Qiuyang Zhan. BlockA2A: Towards secure and verifiable agent-to-agent interoperability, 2025. Blockchain-anchored trust framework using DIDs and smart contracts to verify message authenticity and execution integrity in A2A interoperability; hashes interaction artifacts rather than definitions.
- [56] Amjad Fatmi. Faramesh: A protocol-agnostic execution control plane for autonomous agent systems, 2026. Introduces the Action Authorization Boundary (AAB): a non-bypassable, deterministic PERMIT/DEFER/DENY decision over a Canonical Action Representation, placed between agent reasoning and real-world execution.
- [57] Om Tailor. Verifier-bound communication for LLM agents: Certified bounds on covert signaling, 2026. Protocol that gates messages (not agents) into the shared transcript via a verifier-approved predicate, producing certified upper bounds on covert signaling between colluding LLM agents.

- [58] Anthropic. Agent skills specification. <https://agentskills.io/specification>, 2025. Open standard released December 2025 for portable capability definitions via SKILL .md files (YAML frontmatter + Markdown body) with progressive disclosure; adopted by OpenAI, Microsoft, Cursor, and others.
- [59] Claudiu Leoveanu-Condrei. A DbC inspired neurosymbolic layer for trustworthy agent design, 2025. Adapts Design by Contract to LLM agents; posits that agents satisfying the same contracts are functionally equivalent.
- [60] Haoyu Wang, Christopher M. Poskitt, and Jun Sun. AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents. ICSE 2026, 2026. Formal semantics for runtime rule enforcement in LLM agents.
- [61] W3C WebAgents Community Group. Webagents community group report on interoperability for agents on the web, 2025. Synthesizes FIPA, BDI, and modern LLM-based agent protocols.
- [62] Martin Josifoski, Lars Klein, Maxime Peyrard, Nicolas Baldwin, Yifei Li, Saibo Geng, Julian Paul Schnitzler, Yuxing Yao, Jiheng Wei, Debjit Paul, and Robert West. Flows: Building blocks of reasoning and collaborating AI, 2024. Frames AI agents as modular “semantic processors” exchanging typed messages.
- [63] Microsoft. NLWeb: Bringing conversational interfaces directly to the web, 2025. Uses Schema.org as semantic foundation for agent-web interaction; every instance exposes an MCP server.